



République Tunisienne

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA
RECHERCHE SCIENTIFIQUE

Université de Carthage

Institut National des Sciences Appliquées et de Technologie

RAPPORT DE STAGE D'INGÉNIEUR

Vector – Conception et développement d'un assistant IA d'entreprise pour l'analyse de données et l'interrogation documentaire

Filière : INSTRUMENTATION ET MAINTENANCE INDUSTRIELLE

Organisme d'accueil : WELYNE – A.I. COMMANDOS

Présenté par :

SABRI Ranim

Encadrant professionnel :

M. Ben Arfa Mohammed

Année Universitaire 2025 – 2026

Résumé

Ce rapport présente les travaux réalisés dans le cadre du stage effectué au sein de l'entreprise Welyne, dans son programme A.I. Commandos, portant sur la conception, le développement et le déploiement de **Vector**, un assistant d'intelligence artificielle d'entreprise destiné à l'analyse de données tabulaires et à l'interrogation documentaire multi-format.

Le projet a été mené de bout en bout, depuis la mise en place de l'infrastructure technique jusqu'au déploiement public d'une démonstration accessible en ligne. Il combine une architecture backend en *FastAPI* (Python) et frontend en *Next.js* (TypeScript), une base de données *PostgreSQL* enrichie de l'extension *pgvector* pour la recherche sémantique, ainsi qu'une intégration des modèles d'*OpenAI* (GPT-4o et embeddings) pour la génération de réponses et l'analyse de contenu.

Vector propose plusieurs fonctionnalités clés : un chat conversationnel multimodal (texte, voix, image), l'ingestion et le profilage automatique de datasets tabulaires, l'interrogation documentaire par génération augmentée par récupération (*RAG*), la constitution de corpus thématiques multi-format, la création d'assistants personnalisés (*personas*), la génération automatique de tableaux de bord d'indicateurs clés, ainsi qu'une gestion complète du compte utilisateur conforme aux exigences de protection des données.

Au-delà du développement fonctionnel, une attention particulière a été portée à la qualité industrielle du produit : plus de 310 tests unitaires backend et 46 tests end-to-end *Playwright* ont été mis en place, une couverture de tests atteignant 75 % a été validée par un audit rigoureux, et un pipeline d'intégration continue *GitLab CI/CD* vert exécute l'ensemble des vérifications à chaque modification. Le déploiement final sur la plateforme *Render.com*, avec une base de données externalisée sur *Neon*, a permis de rendre Vector accessible publiquement via une URL stable.

Mots-clés : Assistant IA, RAG, LLM, FastAPI, Next.js, PostgreSQL, pgvector, OpenAI, Docker, CI/CD, Render, déploiement cloud.

Abstract

This report presents the work carried out during the internship at Welyne, as part of its A.I. Commandos program, focusing on the design, development, and deployment of **Vector**, an enterprise-grade AI assistant dedicated to tabular data analysis and multi-format document querying.

The project was managed end-to-end, from the initial technical infrastructure setup to the public deployment of an online demonstration. It combines a *FastAPI* (Python) backend with a *Next.js* (TypeScript) frontend, a *PostgreSQL* database enhanced with the *pgvector* extension for semantic search, and integration of *OpenAI* models (GPT-4o and embeddings) for content generation and analysis.

Vector offers several key features : a multimodal conversational chat (text, voice, image), the ingestion and automatic profiling of tabular datasets, document querying through Retrieval-Augmented Generation (*RAG*), the creation of multi-format thematic document corpora, custom AI assistants (*personas*), automatic generation of KPI dashboards, as well as comprehensive user account management compliant with data protection requirements.

Beyond functional development, particular attention was paid to industrial-grade product quality : over 310 backend unit tests and 46 *Playwright* end-to-end tests were implemented, a test coverage of 75% was validated through a rigorous audit, and a green *GitLab CI/CD* pipeline executes all verifications on every code change. Final deployment on the *Render.com* platform, with an externalized database on *Neon*, made Vector publicly accessible through a stable URL.

Keywords : AI Assistant, RAG, LLM, FastAPI, Next.js, PostgreSQL, pgvector, OpenAI, Docker, CI/CD, Render, cloud deployment.

Table des matières

Résumé	1
Abstract	2
Nomenclatures et abréviations	8
Introduction générale	15
1 Cadre général du stage	17
1.1 Introduction	17
1.2 Présentation de l'organisme d'accueil	17
1.2.1 Welyne : présentation générale	17
1.2.2 Domaines d'expertise et positionnement sectoriel	18
1.2.3 Le programme A.I. Commandos	18
1.2.4 Positionnement du stage	19
1.3 Contexte et problématique	19
1.3.1 Le paysage des assistants IA d'entreprise	19
1.3.2 Les limites des solutions généralistes du marché	20
1.3.3 Positionnement de Vector	20
1.4 Objectifs du stage	21
1.4.1 Objectif principal	21
1.4.2 Objectifs pédagogiques et techniques mesurables	21
1.4.3 Livrables attendus	21
1.5 Méthodologie de travail	21
1.5.1 Approche par sprints courts	21
1.5.2 Développement assisté par intelligence artificielle	22
1.5.3 Discipline documentaire et communication	22
1.6 Environnement de travail	22
1.6.1 Poste et chaîne d'outils	22
1.6.2 Dépôt de code et cycle de développement	22
1.7 Planning et déroulement du stage	23
1.7.1 Chronologie des grandes phases	23
1.7.2 Répartition indicative du temps	23
1.8 Conclusion	24
2 Étude technique et choix des outils	25
2.1 Introduction	25
2.2 Architecture générale d'un assistant IA moderne	25
2.2.1 Vue d'ensemble	25
2.3 Technologies backend	26

2.3.1	Python 3.11 et FastAPI	26
2.3.2	SQLAlchemy 2.0 et Alembic	26
2.3.3	Pydantic, Uvicorn	26
2.4	Technologies frontend	27
2.4.1	Next.js 16 avec App Router	27
2.4.2	TypeScript, Tailwind CSS v4	27
2.4.3	Bibliothèques d'interface et gestion d'état	27
2.5	Technologies IA et données	28
2.5.1	OpenAI comme fournisseur de LLM	28
2.5.2	Concept clé : Retrieval-Augmented Generation (RAG)	28
2.5.3	Tavily API pour la recherche web	29
2.6	Base de données et persistance	29
2.6.1	PostgreSQL 16 avec pgvector	29
2.6.2	Redis en développement uniquement	29
2.7	Infrastructure et conteneurisation	30
2.7.1	Docker et Docker Compose	30
2.7.2	Nginx comme reverse proxy interne	30
2.8	Contrôle de version, CI/CD, tests et déploiement	30
2.8.1	Git, GitLab et GitLab CI/CD	30
2.8.2	Tests et linting	30
2.8.3	Render.com et Neon	31
2.9	Synthèse des choix techniques	31
2.10	Conclusion	32
3	Conception et développement de Vector	34
3.1	Introduction	34
3.2	Vision produit et cahier des charges fonctionnel	34
3.2.1	Ce que Vector doit permettre à l'utilisateur	34
3.2.2	Principes de conception directeurs	35
3.3	Architecture générale	35
3.3.1	Vue d'ensemble système	35
3.3.2	Flux d'une requête type	35
3.4	Base de données et modèle relationnel	36
3.4.1	Choix de PostgreSQL 16 avec pgvector	36
3.4.2	Modèle conceptuel de données	36
3.4.3	Migrations Alembic	37
3.5	Système d'authentification	37
3.6	Module chat conversationnel multimodal	38
3.6.1	Besoin utilisateur et architecture	38
3.6.2	Illustration : chat général	39
3.6.3	Multimodalité : image et recherche web	39
3.7	Module ingestion et analyse tabulaire	40
3.7.1	Besoin et interface d'upload	40
3.7.2	Profilage automatique	41
3.7.3	Génération automatique de dashboards	41
3.8	Module RAG documentaire	43
3.8.1	Besoin et interface d'upload	43
3.8.2	Pipeline d'ingestion RAG	43

3.8.3	Retrieval, génération et illustration	44
3.9	Modules corpus et personas	44
3.9.1	Corpus multi-documents	44
3.9.2	Personas : assistants personnalisés	45
3.10	Expérience utilisateur transverse	46
3.10.1	Thème clair et sombre	46
3.10.2	Respect du RGPD et partage de conversation	46
3.11	Conclusion	47
4	Qualité, tests et déploiement	48
4.1	Introduction	48
4.2	Stratégie de tests	48
4.2.1	Philosophie générale	48
4.2.2	Pyramide de tests	48
4.2.3	Périmètre couvert	49
4.3	Tests unitaires backend (pytest)	50
4.3.1	Organisation et statistiques	50
4.3.2	Anomalie détectée sur coverage.py : cas d'école méthodologique . .	50
4.4	Tests end-to-end avec Playwright	51
4.4.1	Choix de Playwright et périmètre	51
4.4.2	Régressions détectées et mock LLM déterministe	51
4.5	Intégration continue GitLab	52
4.5.1	Objectif et structure	52
4.5.2	Performance et incident sécurité	52
4.6	Choix de la plateforme de déploiement	53
4.6.1	Contexte du pivot HuggingFace vers Render	53
4.6.2	Architecture cible du déploiement	54
4.7	Mise en œuvre du déploiement	55
4.7.1	Adaptations nécessaires et sécurisation	55
4.7.2	Les cinq pièges de déploiement rencontrés et résolus	55
4.8	Mesures de performance en production	56
4.8.1	Méthodologie de mesure	56
4.8.2	Résultats obtenus	57
4.8.3	Analyse et interprétation	57
4.9	Démonstration publique livrée	57
4.9.1	URL et compte pré-rempli	57
4.9.2	Enseignements sur le déploiement	58
4.10	Conclusion	58
	Conclusion générale	60

Table des figures

1	Logo de l'entreprise Welyne	17
2	Chronologie des grandes phases du stage	23
3	Architecture type d'un assistant IA d'entreprise moderne	25
4	Pipeline RAG détaillé implémenté dans Vector	28
5	Architecture générale de Vector	35
6	Diagramme de séquence d'une requête chat avec RAG	36
7	Diagramme de séquence de l'authentification	38
8	Interface de chat conversationnel	39
9	Analyse d'image dans le chat	39
10	Chat avec activation de la recherche web	40
11	Interface d'upload de dataset	40
12	Dashboard automatique – vue KPI	41
13	Dashboard automatique – vue graphique	42
14	Assistant de personnalisation d'un widget de dashboard	42
15	Interface d'upload de document	43
16	Pipeline d'ingestion documentaire pour le RAG	43
17	Chat RAG avec citation des sources	44
18	Création d'un corpus documentaire	45
19	Création d'un persona personnalisé	45
20	Vector en modes clair et sombre	46
21	Pyramide de tests appliquée à Vector	49
22	Structure du pipeline GitLab CI	52
23	Architecture de déploiement en production	54
24	Page d'accueil de la démonstration publique de Vector	58

Liste des tableaux

1	Fiche d'identité synthétique de Welyne	18
2	Vue d'ensemble des agents du programme A.I. Commandos	19
3	Comparatif des principaux assistants IA généralistes du marché	20
4	Répartition approximative du temps par catégorie d'activité	23
5	Synthèse des choix techniques structurants	32
6	Périmètre des tests par zone fonctionnelle	49
7	Statistiques finales des tests unitaires backend	50
8	Parcours utilisateur couverts par les tests end-to-end	51
9	Métriques de performance du pipeline GitLab	53
10	Comparatif des plateformes de déploiement évaluées	54
11	Mesures de performance en production	57

Nomenclatures et abréviations

Acronymes et abréviations techniques

Abréviation	Signification
<i>Intelligence artificielle et apprentissage automatique</i>	
IA	Intelligence Artificielle
LLM	Large Language Model – modèle de langage à grande échelle
RAG	Retrieval-Augmented Generation – génération augmentée par récupération
NLP	Natural Language Processing – traitement du langage naturel
GPT	Generative Pre-trained Transformer
ML	Machine Learning – apprentissage automatique
Embedding	Représentation vectorielle numérique d'un texte, image ou entité
Token	Unité de base traitée par un LLM (environ trois quarts d'un mot en français)
Chunk	Fragment de texte issu du découpage d'un document pour la vectorisation
Prompt	Instruction textuelle envoyée à un modèle de langage

Acronymes et abréviations techniques (suite)

Abréviation	Signification
<i>Développement web et logiciel</i>	
API	Application Programming Interface – interface de programmation applicative
REST	REpresentational State Transfer – style architectural pour les APIs
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure – version chiffrée du HTTP via TLS
JSON	JavaScript Object Notation – format d'échange de données
CRUD	Create, Read, Update, Delete – opérations de base sur les données
SDK	Software Development Kit – kit de développement logiciel
UI	User Interface – interface utilisateur
UX	User Experience – expérience utilisateur
SPA	Single Page Application – application web à page unique
SSR	Server-Side Rendering – rendu côté serveur
CSR	Client-Side Rendering – rendu côté client
CSS	Cascading Style Sheets – feuilles de style en cascade
DOM	Document Object Model
<i>Sécurité et authentification</i>	
JWT	JSON Web Token – jeton d'authentification signé cryptographiquement
OAuth	Open Authorization – protocole d'autorisation standard
TLS	Transport Layer Security – protocole de chiffrement des communications
CORS	Cross-Origin Resource Sharing – partage de ressources entre origines
bcrypt	Fonction de hachage cryptographique pour mots de passe
RGPD	Règlement Général sur la Protection des Données

Acronymes et abréviations techniques (suite)

Abréviation	Signification
<i>Bases de données et persistance</i>	
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language – langage de requête structurée
NoSQL	Not Only SQL – bases de données non relationnelles
ORM	Object-Relational Mapping – mapping objet-relationnel
UUID	Universally Unique Identifier – identifiant unique universel
IVFFlat	Inverted File with Flat compression – index de recherche vectorielle
CID	Content Identifier – identifiant basé sur le contenu
<i>Infrastructure, conteneurisation et déploiement</i>	
CI	Continuous Integration – intégration continue
CD	Continuous Deployment – déploiement continu
DevOps	Development & Operations – pratiques unifiant développement et opérations
PaaS	Platform as a Service – plateforme en tant que service
IaaS	Infrastructure as a Service – infrastructure en tant que service
SaaS	Software as a Service – logiciel en tant que service
VM	Virtual Machine – machine virtuelle
CDN	Content Delivery Network – réseau de distribution de contenu
DNS	Domain Name System – système de noms de domaine
<i>Tests et qualité logicielle</i>	
E2E	End-to-End – tests de bout en bout
TDD	Test-Driven Development – développement piloté par les tests
BDD	Behavior-Driven Development – développement piloté par le comportement
QA	Quality Assurance – assurance qualité
KPI	Key Performance Indicator – indicateur clé de performance
MR	Merge Request – demande de fusion (équivalent Pull Request)

Acronymes et abréviations techniques (suite)

Abréviation	Signification
<i>Communication temps réel</i>	
SSE	Server-Sent Events – événements envoyés par le serveur
WebSocket	Protocole de communication bidirectionnelle temps réel
ASGI	Asynchronous Server Gateway Interface – interface serveur asynchrone Python
WSGI	Web Server Gateway Interface – interface serveur synchrone Python
<i>Formats de fichiers et documents</i>	
PDF	Portable Document Format
DOCX	Document Word au format Office Open XML
PPTX	Présentation PowerPoint au format Office Open XML
XLSX	Feuille de calcul Excel au format Office Open XML
CSV	Comma-Separated Values – valeurs séparées par des virgules
MD	Markdown – langage de balisage léger
YAML	YAML Ain't Markup Language – format de sérialisation lisible
<i>Environnement de développement</i>	
IDE	Integrated Development Environment – environnement de développement intégré
CLI	Command Line Interface – interface en ligne de commande
GUI	Graphical User Interface – interface graphique utilisateur
IP	Internet Protocol – protocole Internet
URL	Uniform Resource Locator – localisateur uniforme de ressource

Technologies et outils utilisés

Technologie	Rôle dans Vector
<i>Langages et frameworks</i>	
Python 3.11	Langage principal du backend (typé statiquement via annotations)
FastAPI	Framework backend asynchrone haute performance
Uvicorn	Serveur ASGI exécutant l'application FastAPI
Pydantic	Validation et sérialisation des données côté backend
SQLAlchemy 2.0	ORM asynchrone pour l'interaction avec PostgreSQL
Alembic	Système de gestion des migrations de base de données
Next.js 16	Framework frontend React avec routage et SSR
TypeScript	Sur-couche typée de JavaScript côté frontend
Tailwind CSS v4	Framework CSS utility-first pour le style
React	Bibliothèque de construction d'interfaces déclaratives
<i>Base de données et cache</i>	
PostgreSQL 16	Base de données relationnelle principale
pgvector	Extension PostgreSQL pour la recherche vectorielle
Redis	Cache en mémoire (utilisé en développement local)
<i>Intelligence artificielle</i>	
OpenAI GPT-4o	Modèle de génération de texte principal
OpenAI GPT-4o-mini	Modèle de génération économique pour tâches légères
text-embedding-3-small	Modèle d'embeddings vectoriels d'OpenAI
Tavily API	Service de recherche web temps réel pour l'assistant

Technologies et outils utilisés (suite)

Technologie	Rôle dans Vector
<i>Infrastructure et déploiement</i>	
Docker	Conteneurisation de l'application et de ses dépendances
Docker Compose	Orchestration multi-services en environnement de développement
Nginx	Reverse proxy interne dans le conteneur de production
Render.com	Plateforme d'hébergement (PaaS) du conteneur en production
Neon.tech	Service PostgreSQL serverless externe hébergeant la base de production
<i>Contrôle de version et CI/CD</i>	
Git	Système de versionning distribué
GitLab	Hébergement du dépôt Git et pipelines CI/CD
GitLab CI/CD	Automatisation des tests, lint et vérifications à chaque push
<i>Tests et qualité</i>	
pytest	Framework de tests unitaires backend Python
coverage.py	Mesure de la couverture de tests du code Python
Playwright	Framework de tests end-to-end pilotant un navigateur réel
ruff	Linteur et formateur rapide pour Python
ESLint	Linteur pour code JavaScript et TypeScript

Technologies et outils utilisés (suite)

Technologie	Rôle dans Vector
<i>Bibliothèques d'interface</i>	
Radix UI	Composants d'interface accessibles et personnalisables
shadcn/ui	Collection de composants React stylisés basés sur Radix
ApexCharts	Bibliothèque de graphiques interactifs pour les dashboards
Recharts	Bibliothèque de graphiques React pour les visualisations
Zustand	Gestion d'état légère côté frontend
TanStack Query	Gestion des requêtes serveur et cache côté frontend
<i>Rate limiting et jobs planifiés</i>	
slowapi	Bibliothèque de rate limiting in-memory pour FastAPI
APScheduler	Ordonnanceur de tâches planifiées in-process
<i>Traitement documentaire</i>	
pypdf	Extraction de texte depuis fichiers PDF
python-docx	Extraction de contenu depuis fichiers Word DOCX
python-pptx	Extraction de contenu depuis présentations PowerPoint PPTX
chardet	Détection automatique d'encodage pour fichiers texte
pandas	Manipulation et analyse de données tabulaires
openpyxl	Lecture et écriture de fichiers Excel XLSX

Introduction générale

L'émergence des grands modèles de langage (*Large Language Models*) a profondément transformé le paysage de l'intelligence artificielle depuis 2022. Des outils comme ChatGPT, Claude ou Gemini ont démontré la capacité des IA génératives à comprendre le langage naturel, à raisonner sur des documents complexes et à assister l'humain dans un large éventail de tâches intellectuelles. Cette révolution technologique s'accompagne d'une demande croissante des entreprises pour des solutions d'intelligence artificielle capables d'exploiter leurs propres données, dans un cadre maîtrisé et adapté à leurs besoins métier.

Cependant, les assistants IA généralistes disponibles sur le marché présentent plusieurs limites face aux exigences du monde professionnel. Ils ne peuvent pas être facilement personnalisés selon un domaine d'expertise précis, ils imposent une dépendance forte à une plateforme unique, et ils soulèvent des questions légitimes concernant la confidentialité des données confiées. Face à ces contraintes, la conception d'assistants IA sur-mesure devient une piste stratégique pour les entreprises souhaitant tirer parti de cette technologie tout en gardant le contrôle sur leurs données et leurs processus.

C'est dans ce contexte que s'inscrit le projet **Vector**, développé au sein de l'entreprise **Welyne** dans le cadre de son programme *A.I. Commandos*. L'objectif de ce stage a consisté à concevoir, développer et déployer, de manière autonome et complète, un assistant IA d'entreprise capable de dialoguer avec l'utilisateur en langage naturel, d'analyser des données tabulaires, d'interroger des documents multi-format, et de produire des visualisations d'indicateurs clés à la demande.

Problématique centrale : comment concevoir, développer et déployer un assistant IA d'entreprise complet et industriellement fiable, offrant une expérience utilisateur comparable à celle des solutions commerciales de référence, tout en garantissant la maîtrise technique, la robustesse à long terme et la possibilité d'une évolution future ?

Cette problématique se décline en plusieurs défis techniques et méthodologiques :

1. **Conception d'une architecture évolutive :** bâtir un système intégrant un backend performant, un frontend moderne, une base de données vectorielle et l'orchestration de plusieurs modèles d'IA externes.
2. **Développement fonctionnel complet :** implémenter un ensemble cohérent de fonctionnalités couvrant l'ingestion documentaire, l'analyse de données, la conversation, les assistants personnalisés et la génération de tableaux de bord.
3. **Qualité industrielle :** mettre en place une stratégie de tests robuste (unitaires, intégration, end-to-end), une chaîne d'intégration continue, et une documentation complète permettant la reprise du projet par un tiers.
4. **Déploiement public :** rendre l'application accessible en ligne via une démonstration stable, sécurisée, hébergée dans une architecture cloud maîtrisée.

Pour répondre à cette problématique et présenter les travaux réalisés, ce rapport s'articule en quatre chapitres complémentaires :

- Le **premier chapitre** présente le cadre général du stage : l'organisme d'accueil, le contexte du projet, les objectifs visés, la méthodologie de travail adoptée et l'environnement technique mis en place.
- Le **deuxième chapitre** détaille l'étude technique préalable et la justification des choix d'outils et de technologies. Un glossaire des termes fréquemment utilisés est proposé en préambule pour faciliter la compréhension.
- Le **troisième chapitre** décrit la conception et le développement de Vector fonctionnalité par fonctionnalité : architecture générale, modèle de données, module de chat conversationnel, ingestion documentaire, corpus, personas et tableaux de bord.
- Le **quatrième chapitre** présente la stratégie de qualité mise en place (tests, intégration continue, audits), ainsi que le déploiement final sur une plateforme d'hébergement publique avec les mesures de sécurisation associées.

Une conclusion générale dresse le bilan des travaux réalisés, identifie les compétences acquises, discute les difficultés rencontrées et propose des perspectives d'évolution pour le projet Vector.

Chapitre 1

Cadre général du stage

1.1 Introduction

Ce premier chapitre situe le stage dans son contexte général et fournit les clés de lecture nécessaires pour appréhender les chapitres techniques qui suivent. Il présente successivement l'organisme d'accueil et le programme dans lequel s'est inscrit le projet, le paysage technologique dans lequel Vector prend place, les objectifs poursuivis, la méthodologie de travail retenue et l'environnement technique mis en place. Le chapitre 2 détaillera ensuite les choix technologiques, le chapitre 3 la conception et le développement, et le chapitre 4 les efforts de qualité et de déploiement.

1.2 Présentation de l'organisme d'accueil

1.2.1 Welyne : présentation générale

Welyne est une entreprise tunisienne de services et conseil en informatique, fondée en 2017 et spécialisée dans le développement de solutions digitales sur mesure. Sa mission est d'accompagner les entreprises dans leur transformation digitale en transformant leurs besoins métier en solutions technologiques concrètes.



FIGURE 1 – Logo de l'entreprise Welyne

L'entreprise compte entre 11 et 50 employés, positionnement qui lui permet de conjuguer la réactivité d'une structure agile avec la capacité de mener à bien des projets d'ampleur significative.

Caractéristique	Valeur
Nom	Welyne
Année de fondation	2017
Siège	Tunisie
Effectif	11 à 50 employés
Nature de l'activité	Services et conseil en informatique
Mission	Accompagner la transformation digitale des entreprises
Marchés	Tunisie, Europe, Moyen-Orient, Afrique

TABLE 1 – Fiche d'identité synthétique de Welyne

1.2.2 Domaines d'expertise et positionnement sectoriel

L'offre technique de Welyne couvre cinq grands domaines : **intelligence artificielle** (contexte du présent stage), **automatisation**, **développement web et mobile**, **ERP et CRM**, et **Business Intelligence**. Cette pluralité traduit une conviction structurante selon laquelle une transformation digitale réussie combine rarement une seule brique technologique.

Welyne intervient auprès de secteurs variés (finance, retail, télécommunications, santé, industrie, éducation) et opère depuis la Tunisie vers l'international, avec une présence en Europe, au Moyen-Orient et en Afrique. Cette dimension internationale se traduit par la prise en compte de la diversité linguistique dans les interfaces et par l'adaptation aux réglementations locales, notamment le RGPD pour les clients européens.

1.2.3 Le programme A.I. Commandos

Le programme *A.I. Commandos* est une initiative interne de Welyne dédiée au développement accéléré de compétences et de produits dans le domaine de l'intelligence artificielle. Sa philosophie repose sur quatre principes directeurs : autonomie et responsabilisation, méthodologie orientée sprints courts, culture de la démonstration, et développement assisté par intelligence artificielle.

Le programme se structure en plusieurs projets parallèles, chacun incarné par un *agent* spécialisé destiné à répondre à un cas d'usage précis (tableau 2).

Agent	Positionnement
Vector	Analyse de données tabulaires, interrogation documentaire, dashboards. (<i>Objet du présent stage.</i>)
Nova	Agent dédié aux workflows générationnels et créatifs.
Sentinel	Agent orienté sécurité et supervision.
Ledger	Agent spécialisé sur les traitements comptables et financiers.
Scout	Agent d’exploration et de veille informationnelle.
Motion	Agent lié aux flux et à l’automatisation de processus.
Hunter	Agent d’investigation et de recherche ciblée.
Echo	Agent de communication et d’interaction multicanal.

TABLE 2 – Vue d’ensemble des agents du programme A.I. Commandos

Vector occupe une place structurante dans ce catalogue : il combine deux compétences transversales très demandées en entreprise, l’analyse de données tabulaires et l’interrogation documentaire par recherche sémantique.

1.2.4 Positionnement du stage

Ce stage s’inscrit dans le cadre de la formation d’ingénieur à l’Institut National des Sciences Appliquées et de Technologie (INSAT), filière Instrumentation et Maintenance Industrielle. Il a été effectué au sein du programme A.I. Commandos de Welyne durant l’été 2026, sur une période de plusieurs semaines de développement intensif.

Le choix d’un stage en développement logiciel intensif orienté intelligence artificielle, dans une filière traditionnellement orientée vers l’ingénierie physique, est motivé par la conviction que les compétences en développement et en science des données sont désormais transversales à l’ensemble des métiers d’ingénieur.

1.3 Contexte et problématique

1.3.1 Le paysage des assistants IA d’entreprise

Depuis la sortie publique de ChatGPT en novembre 2022, le marché des assistants IA a connu une croissance exponentielle [1, 2], à la fois en nombre d’acteurs, en qualité des modèles et en diversité des cas d’usage. Plusieurs acteurs majeurs se sont positionnés sur ce segment, chacun avec ses propres propositions de valeur.

Assistant	Éditeur	Points forts	Cible
ChatGPT	OpenAI	Large adoption, écosystème étendu, Custom GPTs	Grand public, entreprises
Claude	Anthropic	Fenêtre de contexte étendue, sécurité, projets partagés	Développeurs, entreprises
Gemini	Google DeepMind	Intégration Google Workspace, multimodal natif	Utilisateurs Google
Copilot	Microsoft	Intégration native Office 365	Utilisateurs Microsoft
Mistral	Mistral AI	Souveraineté européenne, modèles open source	Entreprises européennes

TABLE 3 – Comparatif des principaux assistants IA généralistes du marché

Cette abondance masque cependant plusieurs limites structurelles qui deviennent problématiques dans un contexte professionnel exigeant.

1.3.2 Les limites des solutions généralistes du marché

L'analyse des retours d'expérience des entreprises ayant adopté ces assistants dans des contextes métier fait émerger cinq grandes catégories de limites.

- **Dépendance à un fournisseur unique** : les décisions produites d'un éditeur externe (tarifs, dépréciations, changement de comportement des modèles) impactent directement les flux critiques d'une organisation.
- **Confidentialité et souveraineté des données** : les données transmises transitent par des serveurs tiers, généralement hors Union européenne, ce qui pose des questions juridiques dans les contextes sensibles.
- **Personnalisation limitée** : les mécanismes proposés (Custom GPTs, extensions Copilot) ne permettent pas d'intégrer un assistant profondément dans un système d'information existant.
- **Coût à l'échelle** : la facturation par utilisateur devient prohibitive lorsqu'une solution doit être déployée à grande échelle, faisant basculer le calcul économique en faveur d'une solution interne au-delà d'un certain seuil.
- **Uniformité des résultats** : les assistants généralistes ne connaissent pas l'histoire de l'organisation qui les utilise (projets, vocabulaires internes, décisions passées) et produisent des réponses génériques dépourvues d'ancrage contextuel.

1.3.3 Positionnement de Vector

C'est dans ce contexte que Welyne a formulé le besoin d'un assistant IA d'entreprise propriétaire, capable d'adresser simultanément trois catégories d'usages : la conversation

générale sur base de connaissances internes, l'analyse de données tabulaires et l'interrogation par recherche sémantique sur des corpus documentaires métier.

Vector se veut une réponse concrète à ces limites, conçu autour de cinq principes structurants : cloisonnement strict des données (chaque utilisateur n'accède qu'à ses propres ressources), personnalisation via des *personas* (assistants spécialisés associant instructions et corpus), hébergement maîtrisé (déploiable en autonomie), transparence des sources (citation systématique des documents d'origine) et respect du RGPD (suppression totale des données possible à tout moment).

1.4 Objectifs du stage

1.4.1 Objectif principal

L'objectif principal du stage consiste à concevoir, développer et déployer Vector de bout en bout, en autonomie complète, depuis la mise en place initiale de l'infrastructure technique jusqu'à la mise en ligne d'une démonstration publique accessible via une URL stable.

1.4.2 Objectifs pédagogiques et techniques mesurables

Le stage vise l'acquisition et le renforcement d'un ensemble structuré de compétences réparties sur cinq axes : maîtrise d'une stack technique moderne (FastAPI, Next.js, PostgreSQL avec pgvector), compréhension et intégration des mécanismes d'IA générative (LLM, RAG, tool calling, streaming), application des bonnes pratiques de développement logiciel (tests, CI/CD, documentation), gestion complète d'un projet en autonomie (cadrage, planification, arbitrages), et déploiement cloud sécurisé d'une application publique.

Au-delà de ces axes qualitatifs, plusieurs objectifs mesurables ont été fixés en amont : couverture de tests unitaires backend supérieure à 70 %, pipeline d'intégration continue fonctionnel, suite de tests end-to-end couvrant les parcours critiques, démonstration publique accessible via une URL stable, documentation technique et utilisateur complète, et compte de démonstration pré-rempli.

1.4.3 Livrables attendus

L'ensemble se traduit par une liste de livrables identifiés dès le cadrage initial : code source complet versionné sur GitLab, documentation technique interne (stratégie de tests, guide de déploiement, guide utilisateur), suite de tests automatisée, pipeline CI/CD fonctionnel, démonstration publique en ligne avec compte pré-rempli, et rapport de stage détaillé.

1.5 Méthodologie de travail

1.5.1 Approche par sprints courts

Le travail a été organisé en cycles de progression structurés autour de la notion de *sprint* : chaque itération, de durée courte (généralement une à quelques journées), porte sur une thématique fonctionnelle précise et se conclut par un incrément livrable et testable.

Cette approche apporte quatre bénéfices concrètement observés : livraison de valeur incrémentale (chaque fin de sprint produit un état démontrable), capacité d'adaptation rapide face aux imprévus techniques, traçabilité fine des décisions grâce aux journaux de bord, et rythme d'accomplissement visible entretenant un moral projet solide.

1.5.2 Développement assisté par intelligence artificielle

Le développement a été mené de bout en bout par mes soins, avec l'appui ponctuel de *Claude Code* (Anthropic), intégré à l'environnement de développement local et utilisé comme aide à la rédaction de code, à l'exploration d'alternatives et à la production de premiers jets de documentation, jamais comme substitut à la conception.

Cet usage suppose une vigilance permanente : les erreurs typiques de ces assistants (invention de fonctions, oubli de cas particuliers) rendent la relecture systématique du code produit non négociable, et cette relecture s'est révélée une source d'apprentissage à part entière.

1.5.3 Discipline documentaire et communication

Chaque sprint a fait l'objet d'un journal de bord documentant les objectifs, les réalisations, les difficultés et les enseignements. Cette pratique s'est révélée un investissement à haute valeur : matière première pour la rédaction du rapport, formalisation forcée des difficultés au moment où elles surviennent, et livrable facilitant l'onboarding d'éventuels successeurs.

Des points réguliers avec l'encadrement Welyne ont permis de valider les orientations produit structurantes, d'arbitrer les choix techniques ayant un impact stratégique ou budgétaire, et de recueillir des retours sur les livrables intermédiaires.

1.6 Environnement de travail

1.6.1 Poste et chaîne d'outils

Le développement a été mené sur un poste Windows disposant des ressources permettant de faire tourner simultanément l'ensemble de la stack en local. La chaîne d'outils quotidienne comprend Docker Desktop (conteneurisation), Python 3.11 (backend), Node.js avec pnpm (frontend), Visual Studio Code (éditeur), PowerShell (terminal), Git et GitLab (versioning), Claude Code comme outil d'aide au développement, et DBeaver (administration PostgreSQL).

1.6.2 Dépôt de code et cycle de développement

Le code source est hébergé sur GitLab dans un dépôt privé du groupe Welyne-Dev. Les modifications suivent un workflow Git rigoureux : branche principale **vector-agent** qui reflète l'état stable, branches de fonctionnalité éphémères pour les sprints structurants, revues systématiques via des *Merge Requests* avant intégration.

Le cycle nominal de développement local passe par le démarrage de la stack via Docker Compose (PostgreSQL + pgvector, cache Redis), le lancement du backend Uvicorn en mode rechargement, le lancement du frontend Next.js en mode développement, l'exécution des tests avant chaque commit, puis le push et la création de la Merge Request.

1.7 Planning et déroulement du stage

1.7.1 Chronologie des grandes phases

Le stage s’est déroulé selon une progression en plusieurs phases fonctionnelles majeures, chacune correspondant à une brique produit distincte (figure 2).

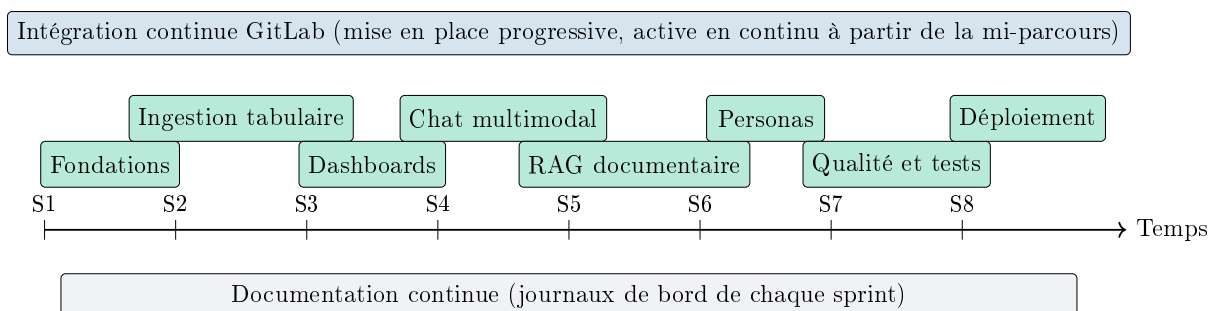


FIGURE 2 – Chronologie des grandes phases du stage

Cette progression n’a pas été strictement séquentielle : plusieurs phases se sont recouvertes, en particulier la documentation menée en continu et l’intégration continue renforcée progressivement.

1.7.2 Répartition indicative du temps

Le tableau 4 donne une répartition approximative du temps investi entre les grandes catégories d’activités, reconstituée a posteriori à partir des journaux de bord et de l’historique Git.

Activité	Part approximative
Développement fonctionnel	50 %
Tests, débogage et corrections	20 %
Documentation et journaux de bord	7 %
Configuration et infrastructure	10 %
Déploiement et itérations de production	7 %
Réunions et communication	6 %

TABLE 4 – Répartition approximative du temps par catégorie d’activité

Deux points ressortent particulièrement : la part cumulée du *non-code* (tests, documentation, configuration, déploiement, communication) atteint environ la moitié du temps total, rappel qu’un projet logiciel est bien davantage qu’une activité de production de code ; et la part significative consacrée aux tests et à la documentation reflète une

décision méthodologique délibérée d'investir dans ces activités dès le début plutôt que de les repousser en fin de projet.

1.8 Conclusion

Ce chapitre a présenté le cadre général du stage : Welyne comme organisme d'accueil, le programme A.I. Commandos comme dispositif de développement produit, le paysage concurrentiel des assistants IA généralistes et les limites qui ont motivé la construction d'une solution propriétaire, les objectifs poursuivis, la méthodologie adoptée et l'environnement technique mis en place.

Le chapitre suivant se concentre sur l'étude préalable des technologies et outils choisis, avec une justification systématique de chaque décision, les alternatives considérées et les compromis assumés.

Chapitre 2

Étude technique et choix des outils

2.1 Introduction

Le développement d'un assistant IA d'entreprise moderne repose sur la combinaison de nombreuses briques technologiques, chacune assumant une responsabilité précise dans l'architecture globale. Le choix de chacune conditionne les performances, la maintenabilité, la sécurité et la viabilité à long terme du produit. Ce chapitre propose une revue systématique des technologies utilisées dans Vector, en explicitant pour chacune son rôle et la justification argumentée de son choix vis-à-vis des alternatives considérées.

2.2 Architecture générale d'un assistant IA moderne

2.2.1 Vue d'ensemble

Un assistant IA d'entreprise contemporain repose sur une architecture distribuée combinant plusieurs couches fonctionnelles, chacune assumant une responsabilité clairement délimitée : une couche interface (expérience utilisateur), une couche métier (règles, orchestration, sécurité), une couche de persistance (données structurées et vecteurs) et une couche IA (délégation aux modèles). La figure 3 en présente le schéma général.

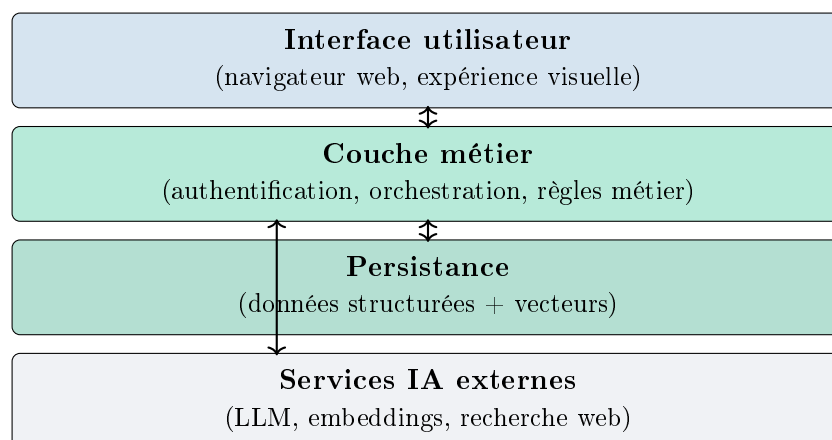


FIGURE 3 – Architecture type d'un assistant IA d'entreprise moderne

Trois choix structurants conditionnent l'ensemble du reste et doivent être pris en

amont : le **langage backend**, le **framework frontend**, et le **fournisseur de modèles IA**. Ces choix sont extrêmement coûteux à revenir en arrière une fois le projet lancé.

2.3 Technologies backend

2.3.1 Python 3.11 et FastAPI

Python 3.11 constitue le langage exclusif du backend Vector. Il gère l'ensemble de la logique métier : requêtes HTTP, orchestration IA, interactions base de données, ingestion documentaire, calculs statistiques.

Justification du choix : l'écosystème IA (SDK OpenAI, LangChain, transformers, pgvector) est distribué en priorité pour Python ; la syntaxe expressive favorise la productivité adaptée au rythme itératif d'un stage ; le support asynchrone natif via `async/await` gère efficacement les workloads I/O-bound d'un backend web ; la communauté et la documentation sont extrêmement larges.

Alternatives écartées : Node.js/TypeScript (écosystème IA moins mature), Go (écosystème IA quasi inexistant), Rust (courbe d'apprentissage prohibitive pour un stage).

FastAPI [3], publié en 2018, est le framework web retenu. Il expose l'ensemble des points d'entrée HTTP de Vector et présente cinq atouts déterminants : performance (parmi les frameworks Python les plus rapides selon les benchmarks TechEmpower), typage statique via Pydantic (validation automatique des entrées/sorties), documentation Swagger UI générée automatiquement à `/docs`, support natif du streaming SSE indispensable pour les réponses IA progressives, et système d'injection de dépendances (Depends) élégant.

Django a été écarté pour sa verbosité et son orientation MVC classique moins adaptée à une API pure moderne ; Flask pour la nécessité d'ajouter manuellement de nombreuses fonctionnalités que FastAPI fournit en standard.

2.3.2 SQLAlchemy 2.0 et Alembic

SQLAlchemy 2.0 [4] (janvier 2023) apporte une refonte majeure avec un typage moderne via `Mapped[]` et un support asynchrone de première classe. Il est utilisé exclusivement en mode async dans Vector, avec le driver `asyncpg` pour PostgreSQL. Alembic gère les évolutions du schéma via des migrations versionnées, applicables ou annulables de manière contrôlée.

Justification : maturité éprouvée en production ; support async natif complet (contrairement à Django ORM dont le support async reste partiel) ; typage moderne intégré avec mypy ; découplé du framework (portabilité des connaissances) ; Alembic est l'outil de migration de facto pour SQLAlchemy avec une bonne intégration.

2.3.3 Pydantic, Uvicorn

Pydantic [5] valide les données via des schémas déclaratifs basés sur les type hints. Chaque requête entrante est validée avant d'atteindre la logique métier, ce qui élimine toute une catégorie de bugs qui se manifesteraient plus tard dans la stack avec des messages beaucoup moins clairs.

Uvicorn est le serveur ASGI ultra-rapide (uvloop + httptools) qui joue le rôle de pont entre le monde HTTP et le monde ASGI de FastAPI. En développement, il tourne

avec rechargement automatique ; en production, en single worker (contrainte RAM du tier Free Render) derrière Nginx.

2.4 Technologies frontend

2.4.1 Next.js 16 avec App Router

Next.js [6] est le framework frontend basé sur React, développé par Vercel. La version 16 introduit le système App Router basé sur les React Server Components, permettant un rendu hybride optimisé. Dans Vector, Next.js gère la navigation, les composants réutilisables, l'état, les appels au backend, et le streaming des réponses IA. En production, il est configuré en mode export statique pour tenir dans les contraintes du tier Free Render.

Justification : maturité et adoption industrielle massive ; écosystème React inégalé en richesse de composants et bibliothèques ; App Router pour une organisation modulaire lisible ; mode export statique précieux pour le déploiement mono-conteneur ; TypeScript de première classe.

Alternatives écartées : Nuxt (écosystème Vue moins riche), SvelteKit (courbe d'apprentissage supplémentaire), Angular (trop lourd pour un projet solo), React sans framework (nécessiterait de recoder routing et build de production).

2.4.2 TypeScript, Tailwind CSS v4

TypeScript apporte au JavaScript un système de types statiques vérifiés à la compilation. Son bénéfice principal pour Vector est la détection précoce des erreurs, particulièrement précieuse lors des refactorings à grande échelle.

Tailwind CSS v4 [7] est un framework CSS « utility-first ». La version 4 introduit un système de *design tokens* déclarés via `@theme inline`, ce qui simplifie considérablement la personnalisation des couleurs et espacements. Trois raisons ont motivé le choix : CSS final extrêmement compact (tree-shaking agressif), évitement du « bikeshedding » sur le nommage des classes (difficulté classique des approches BEM), intégration naturelle avec le système de thèmes clair/sombre via les modificateurs `dark:`.

2.4.3 Bibliothèques d'interface et gestion d'état

Radix UI + **shadcn/ui** combinent primitives comportementales (Radix, headless, accessibilité WCAG stricte) et composants pré-stylés (shadcn, construits avec Tailwind). Material-UI a été écarté pour son style imposé et son bundle size supérieur.

ApexCharts est la bibliothèque de visualisation retenue pour les dashboards générés à partir des datasets. Elle offre une gamme complète de types de graphiques, une esthétique moderne par défaut, et une bonne intégration React.

La gestion d'état frontend sépare deux catégories traitées par des outils spécialisés : **Zustand** pour l'état client léger (préférences, sélections transitoires, thème actif), **TanStack Query** pour le cache serveur (données du backend, avec cache, synchronisation et gestion automatique des états de chargement). Redux a été explicitement rejeté pour sa verbosité (actions, reducers, selectors) au profit de ces outils spécialisés.

2.5 Technologies IA et données

2.5.1 OpenAI comme fournisseur de LLM

OpenAI [8] propose plusieurs modèles utilisés dans Vector : **GPT-4o** [9] (génération des réponses, analyse d'images, génération d'instructions de dashboards), **GPT-4o-mini** (tâches moins critiques et plus fréquentes : titres de conversations, détection sémantique de colonnes), et **text-embedding-3-small** [10] (vectorisation des documents pour le RAG, dimension 1536).

Justification : qualité éprouvée en français; écosystème mature (SDK Python officiel, documentation extensive); multimodalité native (GPT-4o accepte les images sans traitement supplémentaire); contrôle des coûts (facturation à l'usage, plafonds mensuels configurables); streaming stable.

Alternatives écartées : Anthropic Claude (qualité comparable mais pas d'offre gratuite significative pour prototyper), Google Gemini (SDK Python moins mature en français), modèles open source via Groq/Together.ai (qualité française moins prédictible), modèles auto-hébergés (incompatibles avec la contrainte de coût nul).

2.5.2 Concept clé : Retrieval-Augmented Generation (RAG)

Le RAG [11] est le pattern architectural qui permet à un modèle de langage – typiquement une architecture Transformer [12] de la famille GPT [13] – d'exploiter une base documentaire personnalisée sans *fine-tuning*. Le principe : au lieu de demander au modèle de « mémoriser » des documents, on lui fournit les fragments pertinents au moment de la question, dans le prompt.

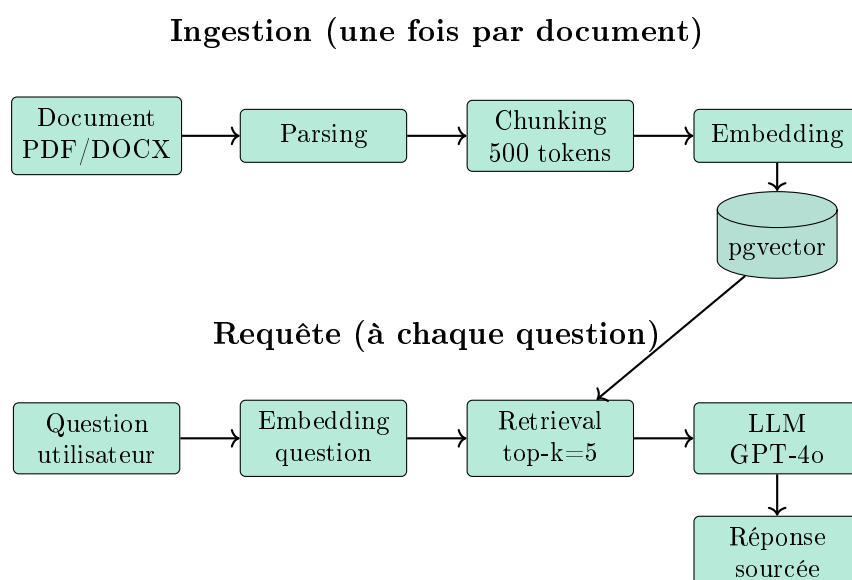


FIGURE 4 – Pipeline RAG détaillé implémenté dans Vector

Ce pattern présente des avantages majeurs sur le *fine-tuning* : mise à jour instantanée à chaque nouveau document, coût opérationnel très faible (pas de GPU dédiée), transparence des sources (traçabilité complète), respect naturel des droits d'auteur.

2.5.3 Tavily API pour la recherche web

Tavily est un service de recherche web spécifiquement conçu pour les cas d'usage IA. Il retourne les résultats sous forme structurée directement exploitable par un LLM. Dans Vector, il est invoqué lorsque l'utilisateur active le toggle « Web ». Une alternative auto-hébergée basée sur SearXNG a été évaluée pour la souveraineté (aucune donnée envoyée à un tiers), mais nécessite un hébergement dédié et un tuning fin — elle reste en réserve.

2.6 Base de données et persistance

2.6.1 PostgreSQL 16 avec pgvector

PostgreSQL [14] est le SGBD relationnel qui héberge toutes les données structurées de Vector : utilisateurs, sessions, conversations, messages, datasets, documents, personas, corpora, dashboards. En production il est hébergé chez Neon (Postgres serverless managé) ; en développement il tourne dans un conteneur Docker local.

Justification : maturité de plus de 25 ans en production critique ; conformité stricte aux standards SQL (portabilité assurée) ; support des types avancés (JSON/JSONB, arrays, extensions natives) ; transactions ACID indispensables pour la cohérence en cascade (suppression utilisateur = suppression atomique de toutes ses ressources) ; extension pgvector installable comme un module natif plutôt que comme un service séparé.

MongoDB a été rejeté explicitement pour l'absence de relations formelles entre documents, qui aurait rendu l'implémentation du cloisonnement multi-utilisateurs beaucoup plus fragile.

pgvector [15] ajoute à PostgreSQL le type `vector` et les opérations de recherche par similarité. Deux versions coexistent dans le projet : l'extension côté serveur PostgreSQL (version **0.8.6** sur Neon, vérifiée par `SELECT extversion FROM pg_extension`) et le client Python (bibliothèque `pgvector-python` version **0.3.6** pinnée dans `pyproject.toml`).

L'avantage majeur de pgvector est la **colocalisation** : la recherche vectorielle réside dans la même base que les données relationnelles, évitant l'ajout d'un service dédié et garantissant la cohérence transactionnelle entre métadonnées et vecteurs. Pinecone (coût récurrent, vendor lock-in), Weaviate/Chroma/Qdrant (service supplémentaire à opérer) et Elasticsearch (overkill pour du RAG pur) ont été écartés.

2.6.2 Redis en développement uniquement

Redis est utilisé en développement local comme broker Celery et stockage éphémère pour le rate limiting. En production sur Render, il a été supprimé pour tenir dans les 512 MB de RAM du tier Free : le rate limiting est assuré par `slowapi` (compteurs en RAM Python) et les tâches planifiées par `APScheduler` (jobs in-process), sans dégradation fonctionnelle perceptible.

2.7 Infrastructure et conteneurisation

2.7.1 Docker et Docker Compose

Docker [16] permet d'encapsuler l'application et ses dépendances dans un conteneur reproductible. Docker Compose orchestre plusieurs conteneurs interdépendants via YAML.

En développement, quatre services sont orchestrés : **vector-db** (PostgreSQL + pgvector), **vector-redis** (cache), backend FastAPI, et frontend Next.js en mode dev. La stack démarre en une seule commande et garantit un environnement identique pour tous les développeurs.

En production Render, un unique conteneur mono-conteneur embarque backend et frontend derrière Nginx. Ce conteneur est construit selon un Dockerfile multi-stage qui optimise à la fois la taille de l'image finale et le temps de build.

Justification : reproductibilité local/prod éliminant les bugs « ça marche chez moi », isolation des dépendances (versions figées dans l'image), portabilité (n'importe quel hébergeur Docker), écosystème mature.

2.7.2 Nginx comme reverse proxy interne

Dans le mono-conteneur Render, Nginx [17] route les requêtes `/api/*` vers Uvicorn (backend) et sert directement les fichiers statiques du frontend pour les autres URLs. Cette architecture expose un unique port HTTP (10000 attendu par Render) tout en séparant les responsabilités. L'empreinte mémoire de Nginx est minimale (quelques dizaines de MB), ce qui laisse la majorité des 512 MB pour Uvicorn.

2.8 Contrôle de version, CI/CD, tests et déploiement

2.8.1 Git, GitLab et GitLab CI/CD

Le workflow adopté suit le modèle *feature branch + merge request* : chaque fonctionnalité développée sur une branche dédiée, testée localement, puis proposée à l'intégration via une Merge Request qui déclenche le pipeline CI. Ce workflow garantit que chaque changement passe par le pipeline CI (qui bloque en cas de régression) et maintient un historique lisible.

GitLab plutôt que GitHub : le dépôt appartient à un groupe organisationnel Welyne déjà hébergé sur GitLab, et l'intégration native de la CI/CD [18] (sans service tiers) simplifie l'ensemble. Le pipeline est piloté par `.gitlab-ci.yml` versionné avec le code — la configuration ne dérive pas entre les branches, les logs sont conservés et les artefacts téléchargeables.

2.8.2 Tests et linting

pytest [19] est le framework de tests unitaires backend, préféré à **unittest** pour sa syntaxe expressive. Il est utilisé avec plusieurs plugins clés : **pytest-asyncio** (tests async), **pytest-cov** (couverture), **pytest-xdist** (parallélisation optionnelle).

Playwright [20] (Microsoft) est le framework de tests end-to-end, capable de piloter Chromium, Firefox et WebKit. Choisi plutôt que Cypress principalement pour la meilleure

gestion des scénarios multi-domaines et multi-onglets et pour l'API plus moderne. Selenium écarté pour son API vieillissante.

Le linting utilise **ruff** (Python, écrit en Rust, extrêmement rapide, remplace flake8+isort+pyupgrade) et **ESLint avec preset TypeScript** (frontend), afin de faire respecter automatiquement les principes de lisibilité et de simplicité du code [21].

2.8.3 Render.com et Neon

Render.com [22] est la plateforme cloud (PaaS) retenue après un pivot depuis HuggingFace Spaces motivé par le changement de politique tarifaire de juillet 2026 (Docker devenu payant). Les alternatives évaluées (Fly.io, Railway, Vercel, Google Cloud Run) sont détaillées au chapitre 4. Render a été retenu pour la combinaison gratuité complète (tier Free sans carte bancaire), support Docker de qualité industrielle, et intégration transparente avec GitLab pour l'auto-déploiement sur push.

Neon [23] est le service Postgres serverless externe. Tier gratuit avec pgvector préinstallé, modèle d'auto-suspend économe, région Frankfurt colocalisée avec Render. Le tier Free Render n'offre pas de PostgreSQL managé, ce qui rend Neon incontournable dans cette architecture.

2.9 Synthèse des choix techniques

Le tableau 5 récapitule les choix techniques structurants et leur justification résumée.

Catégorie	Choix retenu			Justification principale
Backend	Python	3.11	+	Écosystème IA mature, async natif, docs auto
Frontend	Next.js	16	+	Modernité, typage, performance, export statique
Style	Tailwind CSS	v4		Utility-first, tree-shaking, thèmes clair/sombre
Composants UI	Radix UI	+ shadcn/ui		Accessibilité + personnalisation
Visualisation	ApexCharts			Esthétique moderne, gamme complète
État frontend	Zustand	+ TanStack Query		Séparation état client / cache serveur
Base de données	PostgreSQL	16	+	Robustesse, colocalisation vectorielle pgvector
ORM	SQLAlchemy	2.0		Typage moderne, async natif
LLM	OpenAI	GPT-4o / mini		Qualité française, multimodal, SDK mature
Embeddings	OpenAI	text-embedding-3-small		Bon compromis coût/qualité, dim. 1536
Recherche web	Tavily API			Retour structuré adapté aux LLMs
Conteneurisation	Docker	+ Docker Compose		Reproductibilité local/prod
Reverse proxy	Nginx			Maturité, empreinte mémoire minimale
CI/CD	GitLab CI/CD			Intégration native au dépôt
Tests unitaires	pytest	+ coverage.py		Standard Python, riche écosystème
Tests E2E	Playwright			Sélecteurs robustes, cross-browser
Linting	ruff	+ ESLint		Rapidité, standard de facto
Déploiement	Render.com			Docker natif, tier gratuit stable
DB production	Neon			Postgres serverless avec pgvector

TABLE 5 – Synthèse des choix techniques structurants

2.10 Conclusion

Les choix effectués reflètent trois priorités constantes : la **qualité et la maturité** des technologies retenues (aucun choix expérimental non éprouvé), la **cohérence avec l'écosystème IA** (préférence pour les briques les plus intégrées avec les outils IA modernes), et la **contrainte de coût nul** (chaque brique retenue devait avoir une option gratuite viable, au moins pour la démonstration).

Ces décisions techniques constituent la fondation sur laquelle repose l'ensemble des développements présentés dans le chapitre suivant, qui détaille la conception et

l'implémentation module par module de la solution Vector.

Chapitre 3

Conception et développement de Vector

3.1 Introduction

Ce chapitre présente en détail la conception et le développement de Vector, module par module. Il commence par exposer la vision produit et l'architecture générale, puis décrit la conception de la base de données et le système d'authentification. Il enchaîne ensuite avec la présentation de chacun des grands modules fonctionnels : le chat conversationnel multimodal, l'ingestion et l'analyse de données tabulaires, l'interrogation documentaire par RAG, la constitution de corpus multi-formats, la création d'assistants personnalisés (personas), la génération automatique de tableaux de bord, et enfin l'expérience utilisateur transverse.

Chaque module fait l'objet d'une description en trois volets : le besoin utilisateur adressé, l'architecture technique mise en place, et une illustration visuelle du résultat final via des captures d'écran de la démonstration publique.

3.2 Vision produit et cahier des charges fonctionnel

3.2.1 Ce que Vector doit permettre à l'utilisateur

Vector répond à un besoin d'analyste métier qui souhaite exploiter deux catégories de matière première : ses **données tabulaires** (exports CRM, fichiers Excel de suivi commercial, bases de données extraites en CSV) et ses **documents textuels** (rapports stratégiques, notes internes, livres blancs). Historiquement, ces deux types sont traités par des outils différents, sans possibilité de les mettre en dialogue. L'ambition de Vector est de fournir une interface conversationnelle unique capable d'interroger simultanément ces deux gisements, avec la possibilité de croiser les analyses et d'obtenir des réponses citant précisément leurs sources.

Concrètement, un utilisateur doit pouvoir uploader un dataset tabulaire et voir son profilage automatique, uploader des documents et les regrouper en *corpus* thématiques, poser des questions en langage naturel sur ses données ou ses corpus, recevoir des réponses citées, créer des personas spécialisés associant instructions et corpus, générer automatiquement des tableaux de bord avec KPI et visualisations, partager une conversation via un lien en lecture seule, basculer entre thème clair et sombre, et supprimer complètement son compte à tout moment dans le respect du RGPD.

3.2.2 Principes de conception directeurs

Cinq principes traversent l'ensemble des choix de conception : **cloisonnement strict des données** (vérification systématique côté serveur à chaque requête), **transparence des sources** (citation précise nom de fichier, page, section pour toute information issue d'un document), **transparence des coûts** (calcul et affichage du coût en tokens de chaque réponse), **sobriété visuelle** (hiérarchie claire, options secondaires masquées derrière des menus) et **respect du RGPD** (suppression totale avec cascade delete rigoureuse).

3.3 Architecture générale

3.3.1 Vue d'ensemble système

L'architecture de Vector repose sur une séparation classique en trois couches, complétée par des services externes pour l'IA et la recherche web (figure 5).

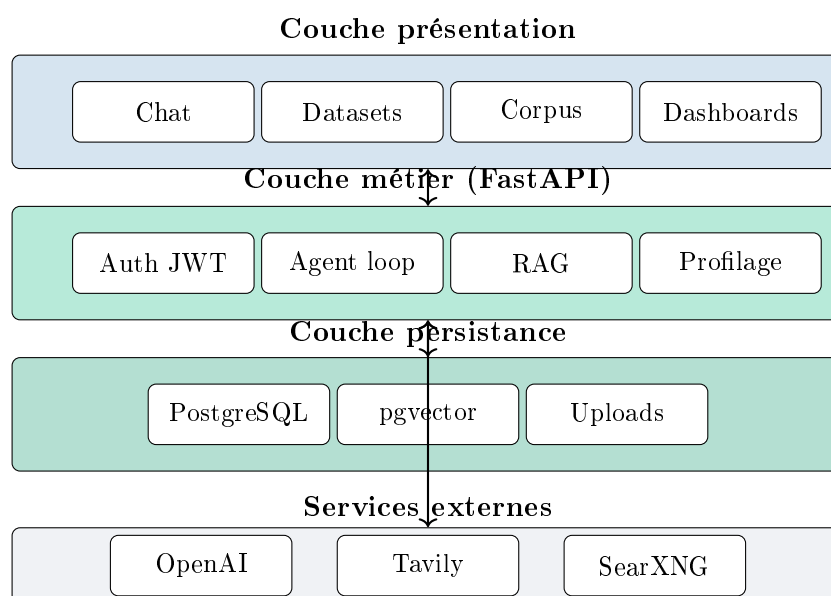


FIGURE 5 – Architecture générale de Vector

Chaque couche a une responsabilité clairement délimitée : la couche présentation (Next.js) gère l'expérience utilisateur sans contenir de logique métier ; la couche métier (FastAPI) authentifie chaque requête, applique les règles métier, orchestre les appels IA et garantit le cloisonnement strict entre utilisateurs ; la couche persistance (PostgreSQL + pgvector) stocke données structurées et vecteurs ; les services externes ne sont invoqués que par la couche métier, garantissant un point unique de contrôle sur les coûts et la sécurité.

3.3.2 Flux d'une requête type

La figure 6 présente le diagramme de séquence d'une requête typique de chat avec activation du RAG.

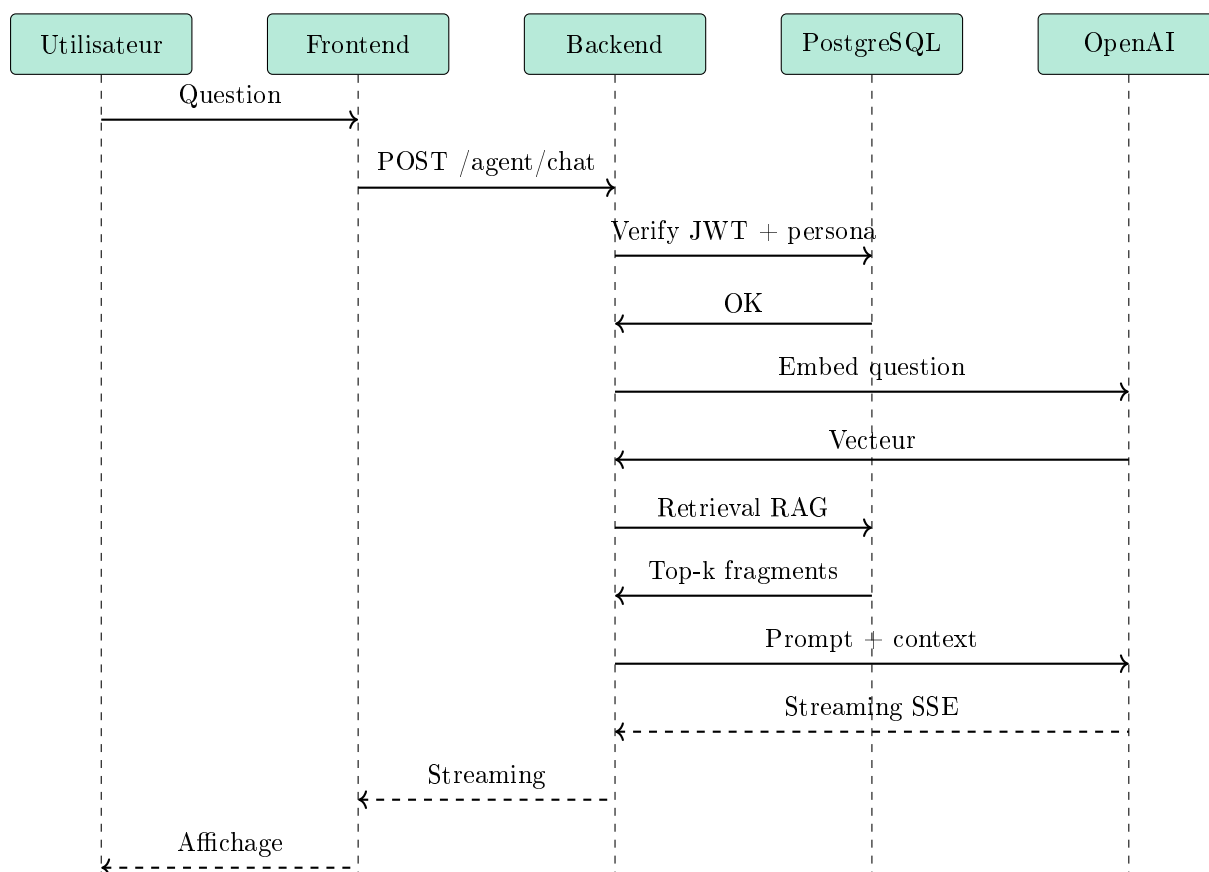


FIGURE 6 – Diagramme de séquence d'une requête chat avec RAG

L'authentification est vérifiée en premier, avant toute autre opération. La question est ensuite vectorisée par un modèle d'embedding, ce vecteur sert à retrouver les fragments les plus proches dans le corpus indexé, ces fragments sont injectés dans le prompt envoyé à OpenAI, et la réponse est streamée en temps réel au navigateur via Server-Sent Events.

3.4 Base de données et modèle relationnel

3.4.1 Choix de PostgreSQL 16 avec pgvector

Le choix de PostgreSQL avec l'extension pgvector est motivé par la volonté d'unifier dans un seul système de stockage l'ensemble des données structurées et des représentations vectorielles nécessaires au RAG. Cette unification simplifie l'architecture opérationnelle (un seul système à provisionner, sauvegarder et surveiller), garantit la cohérence transactionnelle entre les métadonnées d'un document et son vecteur associé, et réduit la surface d'apprentissage à un seul langage.

L'extension pgvector, dans sa version 0.8.6 côté serveur PostgreSQL et 0.3.6 côté client Python, offre des performances suffisantes pour les volumes documentaires typiques du contexte Vector.

3.4.2 Modèle conceptuel de données

Le modèle de données comporte douze tables principales organisées autour de trois blocs fonctionnels : **identité et authentification** (`users`, `tokens_reset`),

conversations et messages (`conversations`, `messages`, `conversation_shares`), **datasets** (`datasets`, `dataset_columns`, `saved_dashboards`, `dashboard_widgets`), **RAG documentaire** (`documents`, `chunks`, `corpora`, `corpus_documents`) et **personas** (`personas`, `persona_corpora`, `agents`).

Le schéma suit rigoureusement les principes de normalisation relationnelle (3NF), avec des clés étrangères strictes garantissant l'intégrité référentielle.

3.4.3 Migrations Alembic

Toutes les évolutions du schéma sont gérées via Alembic. Au terme du stage, l'historique comporte neuf migrations majeures, chacune associée à un sprint fonctionnel (ajout de `corpus_id` sur `conversations`, tables `saved_dashboards`, enrichissement des statistiques colonnes, ajout de `has_seen_welcome`, tokens cost sur `messages`, table `conversation_shares`, table `personas`). Chaque migration est réversible via une méthode `downgrade`, ce qui permet en théorie de revenir à un état antérieur en cas de problème.

3.5 Système d'authentification

L'authentification s'appuie sur des JSON Web Tokens (JWT) [24], signés côté serveur avec une clé secrète stockée dans les variables d'environnement. Ce choix, standard dans les architectures web modernes, présente plusieurs avantages : approche stateless (pas de session à maintenir côté serveur), tokens auto-porteurs, intégration native dans les en-têtes HTTP standards.

Les mots de passe sont hachés avec `bcrypt` (`passlib`), avec un facteur de coût qui rend le calcul volontairement lent (environ 500 ms). Ce coût, apparemment gênant pour la latence, est en réalité le mécanisme de défense principal contre les attaques par force brute : un attaquant qui a compromis la base doit dépenser 500 ms de CPU pour tester chaque combinaison, ce qui rend inopérantes les attaques par dictionnaire massif.

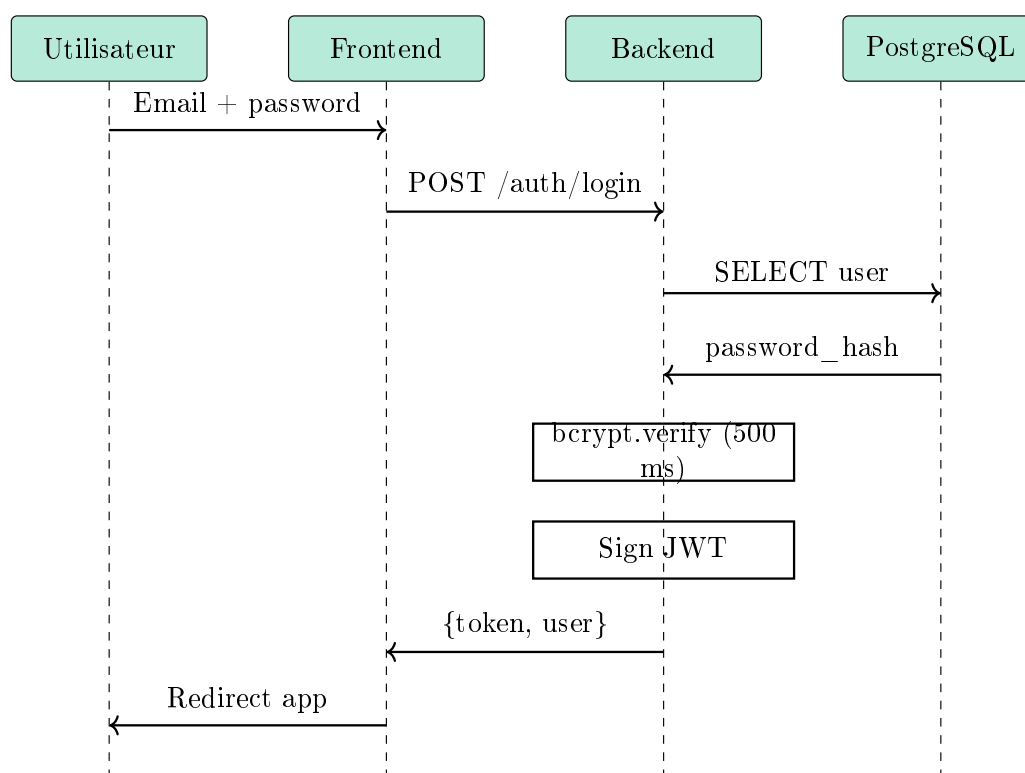


FIGURE 7 – Diagramme de séquence de l'authentification

Le token JWT est stocké côté client dans un cookie HttpOnly (inaccessible au JavaScript, protégeant contre les attaques XSS [25]) et est envoyé automatiquement à chaque requête suivante.

3.6 Module chat conversationnel multimodal

3.6.1 Besoin utilisateur et architecture

Le chat est le point d'entrée principal de l'expérience Vector. C'est par lui que l'utilisateur pose ses questions, reçoit des analyses et interagit avec ses données. Le besoin est celui d'une conversation naturelle en langage courant, avec réception progressive des réponses, historique persistant, et capacité à revenir en arrière pour poursuivre une discussion antérieure.

Le backend expose un endpoint `POST /agent/chat` qui prend en entrée le message, l'identifiant de la conversation, éventuellement un persona et un corpus, et retourne la réponse en mode streaming SSE. Le pipeline interne enchaîne huit étapes : vérification JWT, vérification que la conversation appartient à l'utilisateur, récupération du contexte (historique, persona, corpus), lancement du RAG si un corpus est actif, construction du prompt final, appel OpenAI en streaming, retransmission au frontend en temps réel, et sauvegarde du message complet avec calcul du coût en tokens.

3.6.2 Illustration : chat général

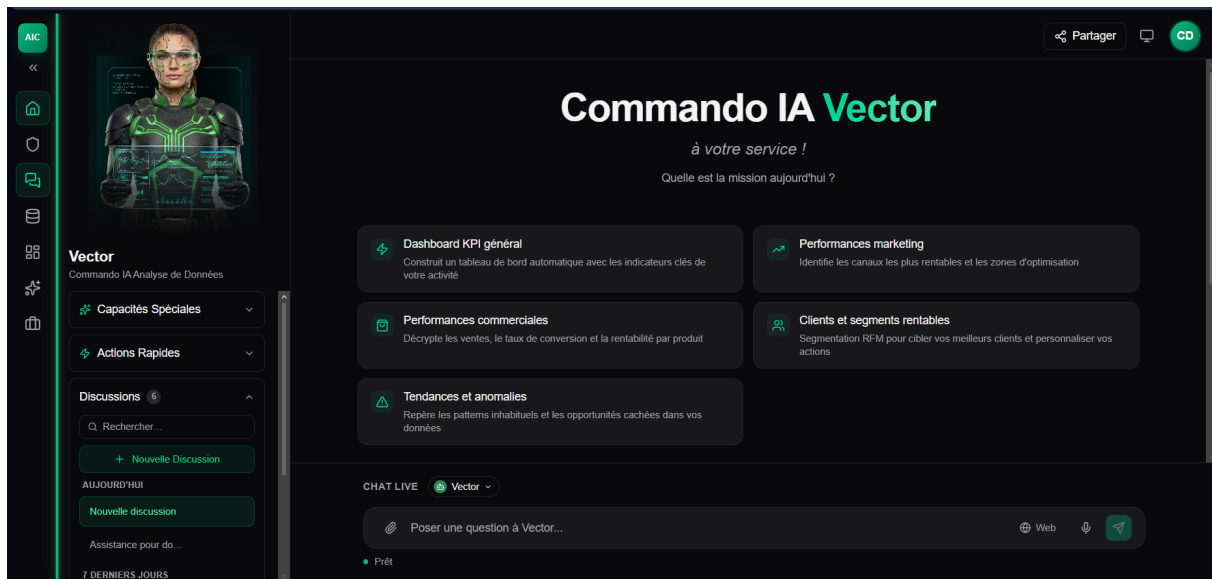


FIGURE 8 – Interface de chat conversationnel

3.6.3 Multimodalité : image et recherche web

Vector supporte l'analyse d'images jointes à un message, en s'appuyant sur les capacités multimodales de GPT-4o. L'utilisateur joint une image directement dans le chat et pose une question à son sujet.

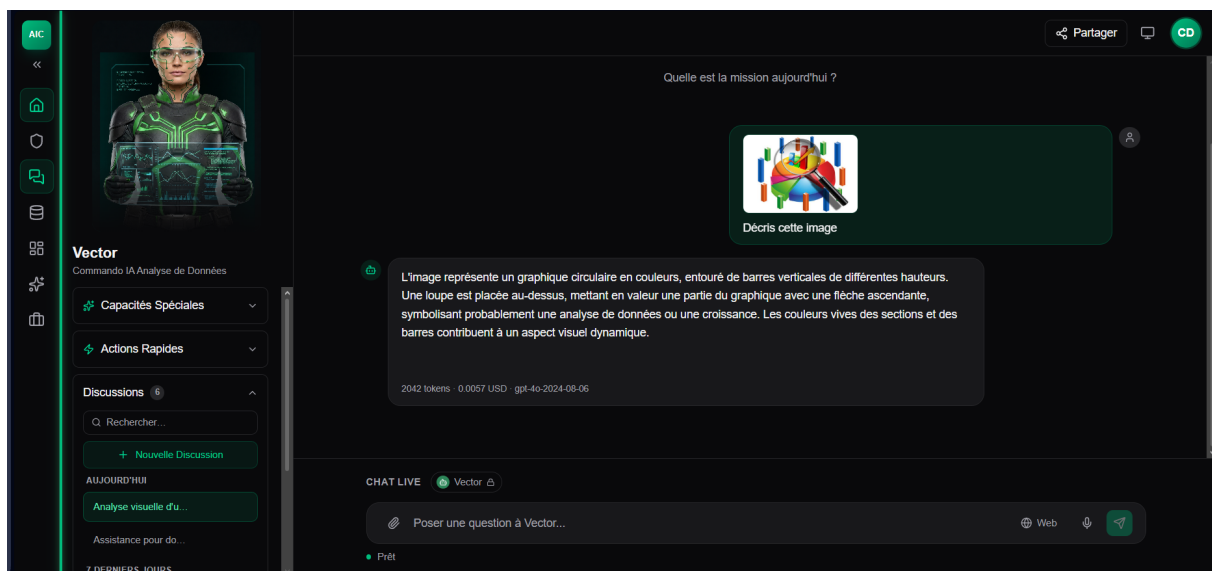


FIGURE 9 – Analyse d'image dans le chat

Un toggle « Web » activable en bas du chat permet à Vector d'aller chercher des informations en ligne pour enrichir sa réponse. La recherche est effectuée via l'API Tavily (SearXNG en fallback), et les sources web sont citées en fin de réponse.

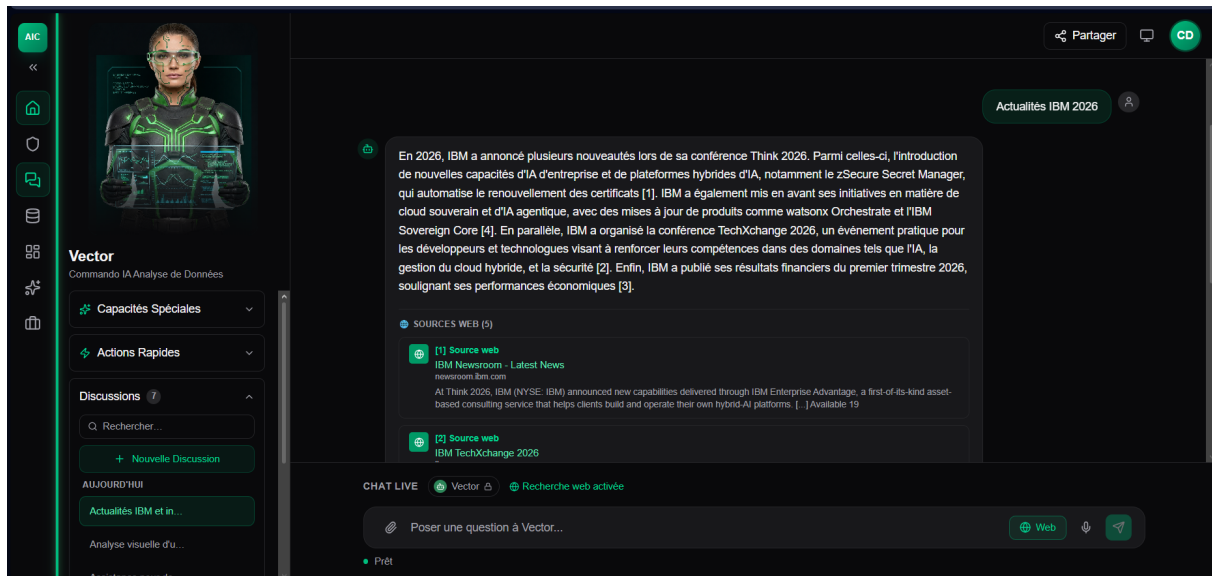


FIGURE 10 – Chat avec activation de la recherche web

3.7 Module ingestion et analyse tabulaire

3.7.1 Besoin et interface d'upload

Un analyste métier reçoit régulièrement des exports tabulaires (extract CRM, rapport financier Excel, base de données extraite). Ces fichiers sont ouverts dans Excel, analysés manuellement, et les tendances restent enfermées dans l'outil. Vector propose une alternative : uploader le fichier, obtenir immédiatement un profilage automatique et un ensemble d'indicateurs, puis pouvoir poser des questions en langage naturel sur le contenu.

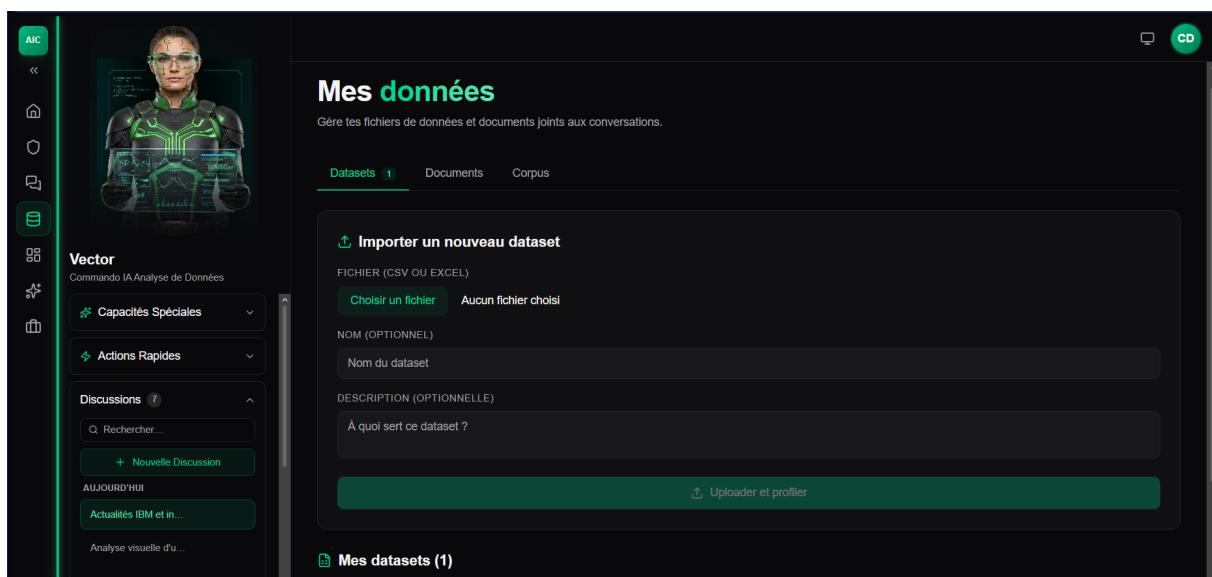


FIGURE 11 – Interface d'upload de dataset

3.7.2 Profilage automatique

Une fois le fichier uploadé, un profilage automatique est effectué en arrière-plan. Il produit quatre types d'informations : **typage automatique des colonnes** (cinq grands types via heuristique combinant analyse des valeurs et nom de colonne), **statistiques descriptives** (valeurs uniques, taux de valeurs manquantes, valeurs d'exemple, statistiques adaptées au type), **analyse sémantique en deux étapes** (identification du contexte métier probable puis détail des tendances observables), et **indicateurs de qualité** (score global entre 0 et 100, liste des problèmes identifiés).

3.7.3 Génération automatique de dashboards

À partir du profilage, Vector génère automatiquement un tableau de bord avec plusieurs indicateurs clés de performance et leurs visualisations. Ce dashboard automatique constitue un point de départ que l'utilisateur peut ensuite personnaliser.

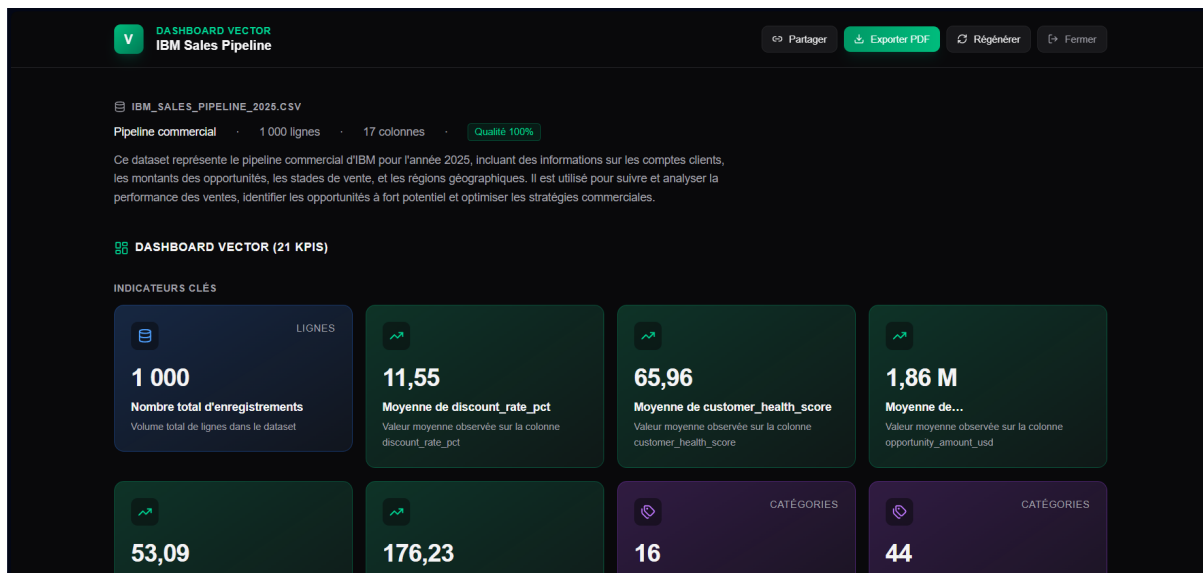


FIGURE 12 – Dashboard automatique – vue KPI

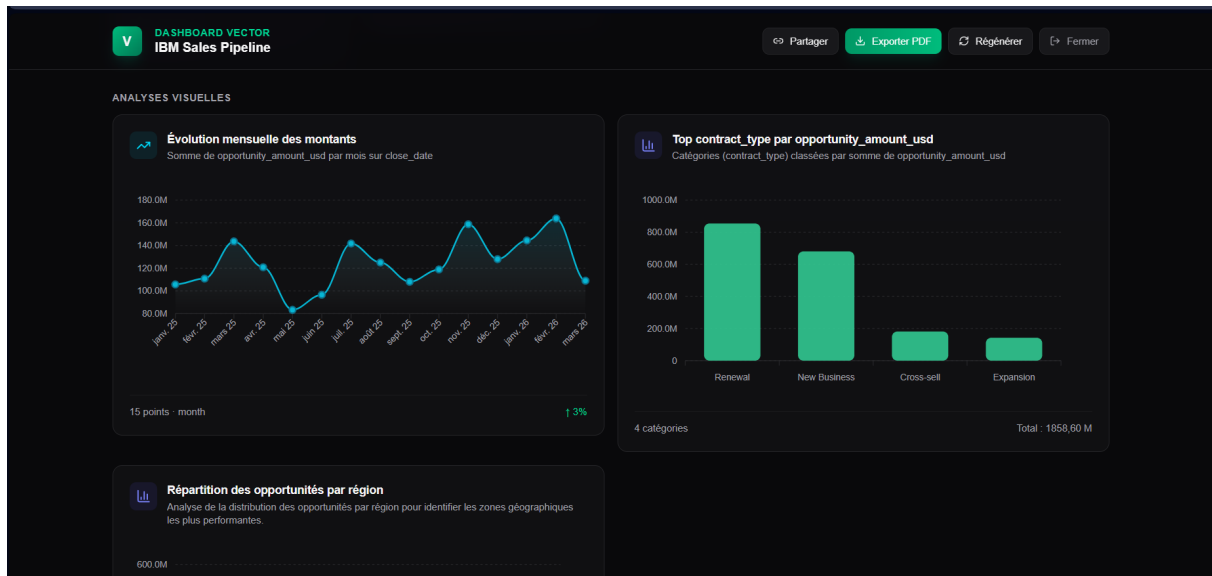


FIGURE 13 – Dashboard automatique – vue graphique

Au-delà des widgets générés automatiquement, l'utilisateur peut enrichir librement son dashboard via un assistant de création en trois étapes : choix du type de widget (KPI, barres, courbe, circulaire, anneau, tableau, radar, nuage de points, heatmap, corrélation, histogramme), sélection des colonnes du dataset à visualiser, puis configuration de l'affichage (titre, agrégation, couleurs). Cette personnalisation permet d'adapter chaque graphique au message précis que l'utilisateur souhaite faire ressortir, plutôt que de se limiter aux visualisations proposées par défaut.

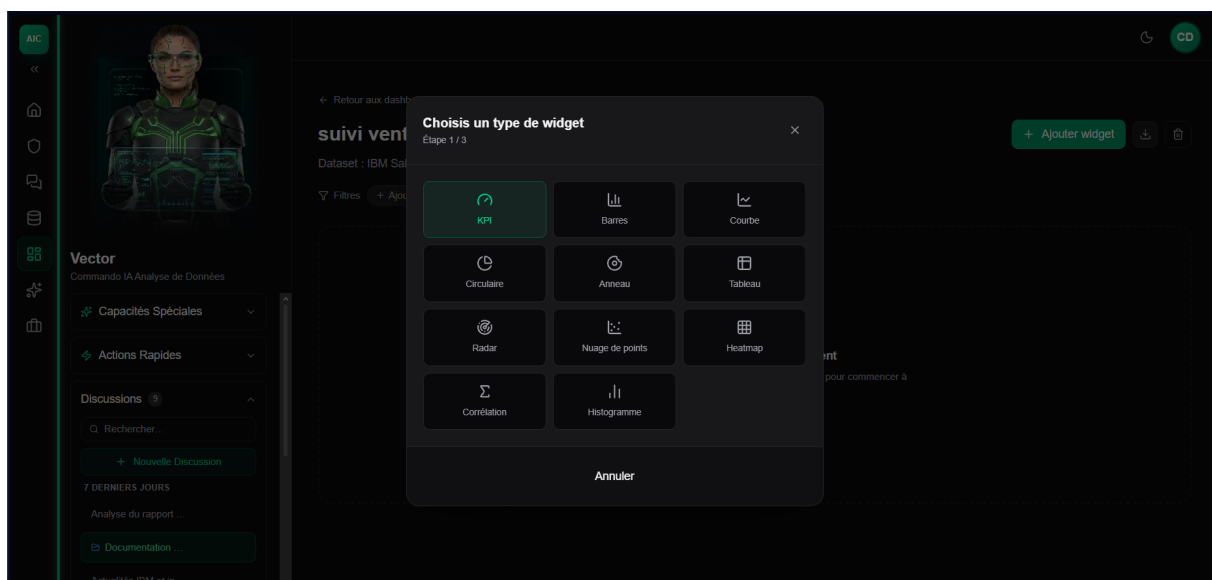


FIGURE 14 – Assistant de personnalisation d'un widget de dashboard

Les visualisations sont produites côté frontend via ApexCharts, choisie pour sa combinaison d'esthétique moderne, de large bibliothèque de types de graphiques et de bonne intégration avec React.

3.8 Module RAG documentaire

3.8.1 Besoin et interface d'upload

Les organisations disposent d'un patrimoine documentaire qui prend de la valeur avec le temps mais dont l'exploitation reste laborieuse. Le RAG (Retrieval-Augmented Generation) apporte une réponse concrète : uploader ces documents dans Vector, les indexer sémantiquement, puis interroger l'ensemble en langage naturel.

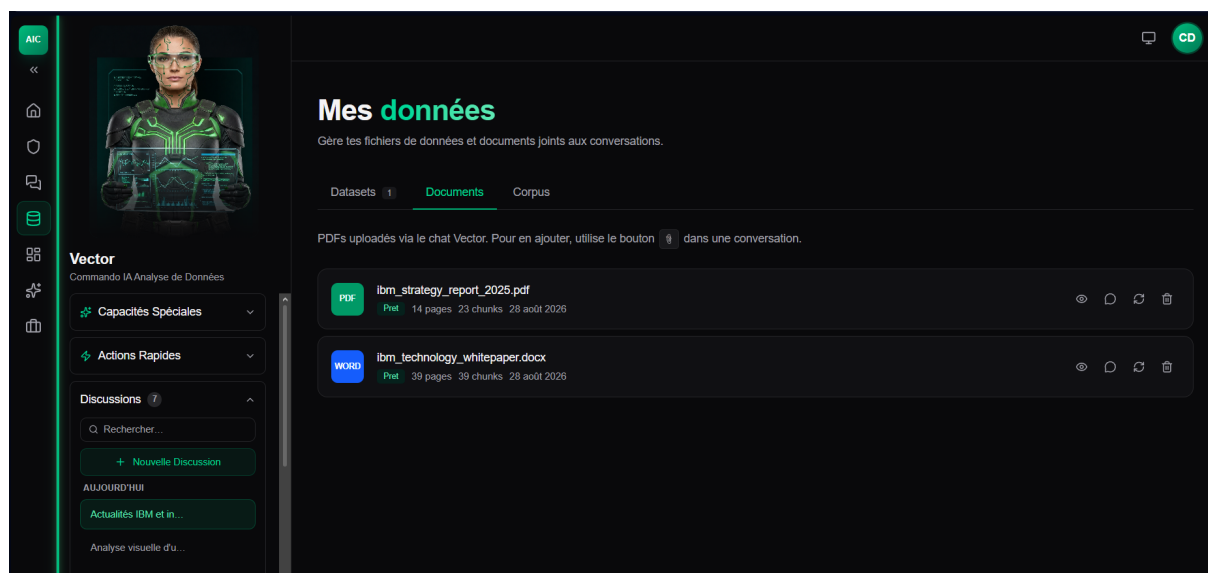


FIGURE 15 – Interface d'upload de document

3.8.2 Pipeline d'ingestion RAG

Une fois un document uploadé, il traverse un pipeline en cinq étapes.

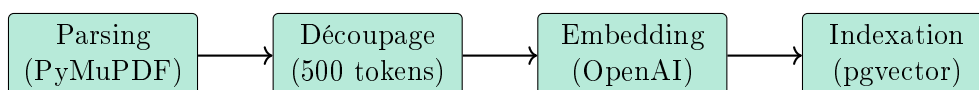


FIGURE 16 – Pipeline d'ingestion documentaire pour le RAG

1. **Parsing** : extraction du texte via PyMuPDF (PDF), python-docx (Word) ou python-pptx (PowerPoint), avec préservation des métadonnées de structure (pages, sections).
2. **Découpage** : le texte est découpé en fragments d'environ 500 tokens avec un chevauchement de 50 tokens entre fragments consécutifs, ce qui évite qu'une idée à cheval sur deux fragments soit perdue.
3. **Embedding** : chaque fragment est converti en vecteur numérique de dimension 1536 via `text-embedding-3-small` d'OpenAI.
4. **Indexation** : les vecteurs sont stockés dans PostgreSQL avec pgvector, avec un index IVFFlat [26] en distance cosinus.
5. **Métadonnées** : chaque fragment conserve un lien vers son document, sa page et sa position, pour permettre la citation des sources.

3.8.3 Retrieval, génération et illustration

Au moment d'une question, la question est convertie en vecteur, les cinq fragments les plus similaires (top-k=5) au-dessus d'un seuil de similarité de 0.7 sont récupérés, puis injectés comme contexte dans le prompt envoyé au modèle. Les fragments sélectionnés sont également retournés au frontend pour affichage sous forme de sources cliquables.

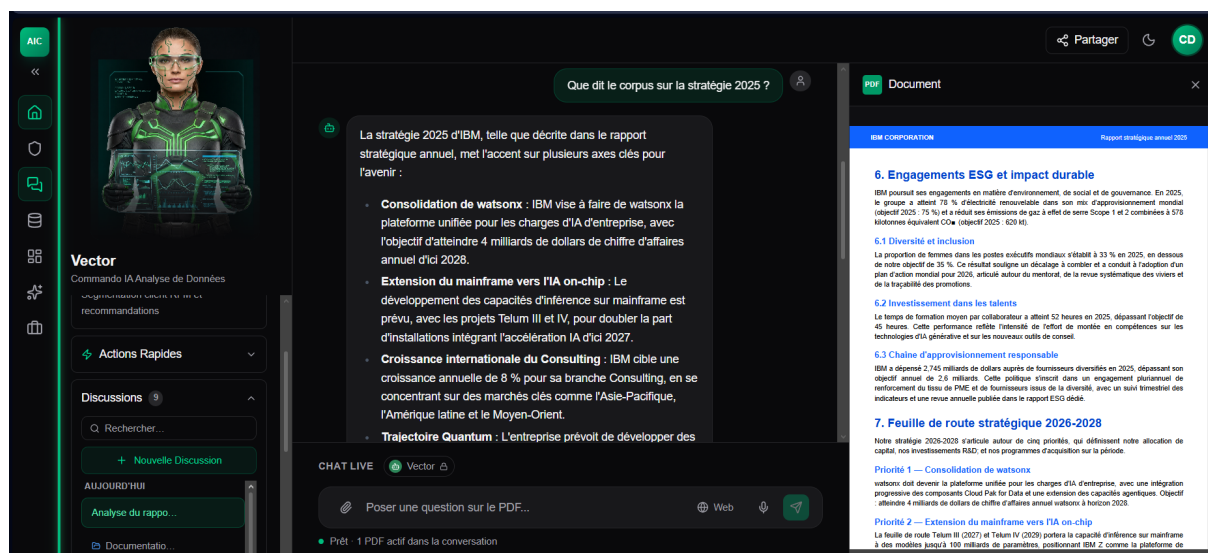


FIGURE 17 – Chat RAG avec citation des sources

3.9 Modules corpus et personas

3.9.1 Corpus multi-documents

Un utilisateur qui a uploadé plusieurs documents veut pouvoir les regrouper thématiquement (« Documentation client A », « Livres blancs concurrents », « Notes stratégiques internes »). Le RAG est alors restreint au corpus sélectionné, ce qui garantit que les réponses ne mélangeront pas des sources appartenant à des périmètres différents.

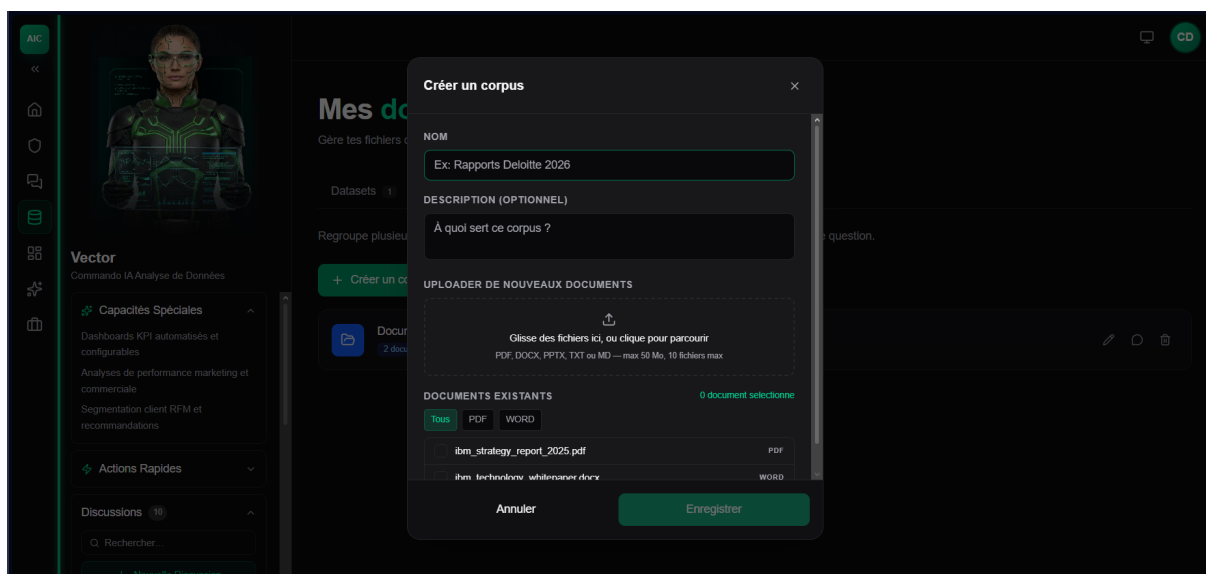


FIGURE 18 – Création d'un corpus documentaire

L'isolation stricte entre corpus est garantie côté serveur : un utilisateur qui pose une question avec un corpus A sélectionné ne verra jamais dans les sources retournées un fragment issu d'un corpus B, même s'il en possède un. Cette vérification est effectuée dans la requête SQL de retrieval, en ajoutant systématiquement une clause `WHERE corpus_id = ?` à la recherche vectorielle.

3.9.2 Personas : assistants personnalisés

Un utilisateur régulier veut pouvoir sauvegarder ses configurations préférées plutôt que de reconfigurer manuellement le contexte à chaque conversation. La notion de *persona* répond à ce besoin : un profil sauvegardé qui combine un jeu d'instructions (« tu es un analyste financier senior chez IBM »), éventuellement un corpus et un dataset associés, et un style de réponse préconfiguré.

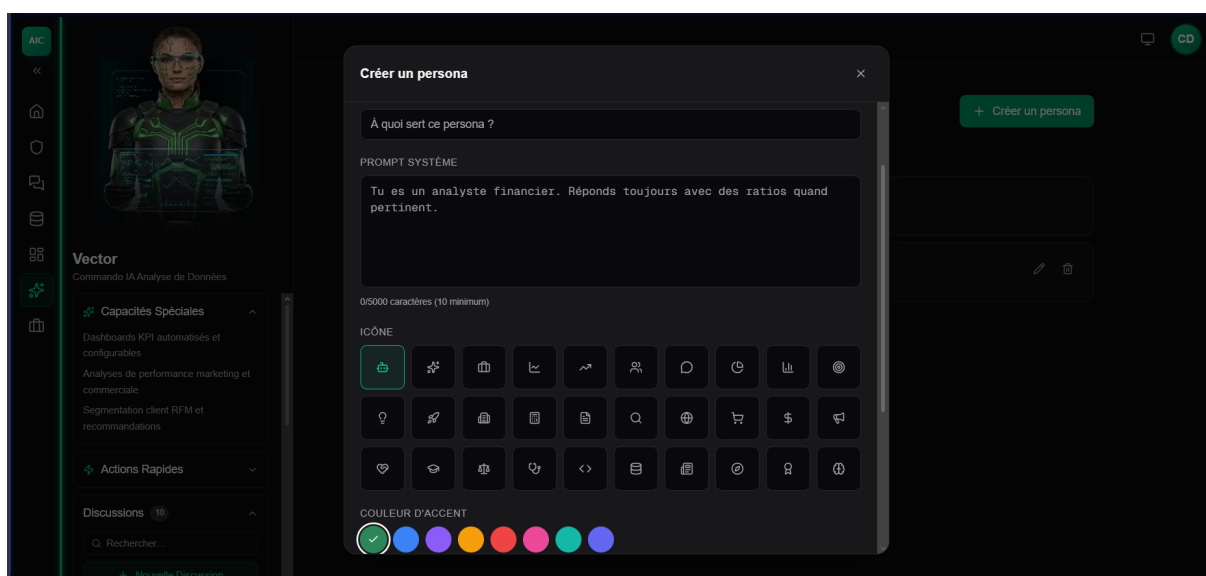


FIGURE 19 – Création d'un persona personnalisé

Vector propose par défaut un persona « Vector » générique, marqué comme persona système, disponible pour tous les utilisateurs. À côté, chaque utilisateur peut créer autant de personas personnalisés qu'il le souhaite, chacun destiné à un cas d'usage précis.

3.10 Expérience utilisateur transverse

3.10.1 Thème clair et sombre

Vector propose une bascule entre thème clair et thème sombre, avec persistance de la préférence utilisateur entre les sessions.

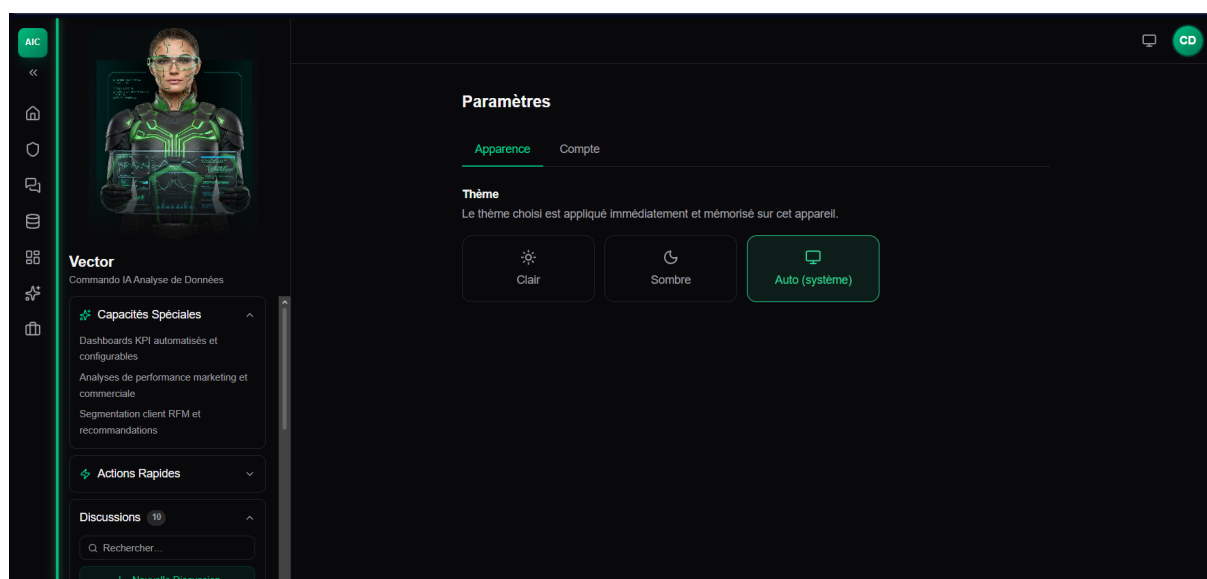


FIGURE 20 – Vector en modes clair et sombre

Le choix technique repose sur la fonctionnalité *dark mode* native de Tailwind CSS v4, avec une classe CSS **dark** appliquée dynamiquement à l'élément racine du document. Toutes les couleurs de l'interface sont définies en deux variantes, et le passage de l'une à l'autre est instantané.

3.10.2 Respect du RGPD et partage de conversation

Le RGPD [27] impose que tout utilisateur puisse supprimer intégralement ses données à tout moment. Vector implémente cette exigence via un mécanisme de cascade delete rigoureux : la suppression d'un utilisateur déclenche automatiquement la suppression de toutes ses ressources, avec vérification par les tests unitaires que rien ne subsiste après suppression.

Un utilisateur peut également générer un lien de partage en lecture seule d'une de ses conversations. Ce lien contient un token opaque non devinable, et l'accès est en lecture seule stricte : impossible pour un consultant sans compte de poursuivre la conversation, d'accéder à d'autres conversations, ou de deviner l'identité de l'utilisateur d'origine.

3.11 Conclusion

Ce chapitre a présenté en détail la conception et le développement de Vector, en parcourant l'ensemble des modules fonctionnels : architecture générale, base de données, authentification, chat conversationnel multimodal, ingestion et analyse tabulaire, RAG documentaire, corpus, personas et expérience utilisateur transverse.

Chaque module a été présenté selon la même grille : besoin utilisateur adressé, architecture technique, illustration visuelle du résultat obtenu sur la démonstration publique. Le chapitre suivant présente les efforts de qualité (tests unitaires, tests end-to-end, intégration continue) et de déploiement (mise en production sur Render.com) qui ont accompagné le développement fonctionnel et permis d'aboutir à une démonstration accessible publiquement.

Chapitre 4

Qualité, tests et déploiement

4.1 Introduction

Le développement fonctionnel présenté au chapitre précédent constitue le cœur du travail, mais il ne suffit pas à faire de Vector un produit industriellement viable. Un logiciel professionnel se distingue par sa **robustesse démontrée** et sa **capacité à être livré en production** sans régression. Ce chapitre présente les efforts de qualité et les travaux de déploiement qui ont accompagné et complété le développement fonctionnel : stratégie de tests (unitaires, end-to-end, mocks déterministes), audits de couverture, pipeline d'intégration continue GitLab, choix et mise en œuvre du déploiement public sur Render, et retour d'expérience sur les mesures de performance en production.

4.2 Stratégie de tests

4.2.1 Philosophie générale

La stratégie de tests de Vector s'inscrit dans une philosophie de *qualité intrinsèque* plutôt que de *qualité vérifiée en fin de projet*. Les tests ont été écrits au fil du développement, en parallèle du code applicatif, avec pour objectif que chaque fonctionnalité nouvelle soit accompagnée de sa couverture de test dès son introduction dans la base de code.

Cette approche apporte quatre bénéfices concrètement observés : confiance dans les refactorings [28] (restructuration sans crainte de régression silencieuse), documentation exécutable (les tests montrent comment chaque fonction est censée se comporter), détection précoce des régressions (identification dans la minute), et rythme de développement soutenu (élimination des phases de débogage tardives).

4.2.2 Pyramide de tests

La stratégie s'organise autour de la classique pyramide en trois niveaux (figure 21), chaque niveau adressant un type de risque distinct avec un compromis différent entre volume, rapidité et fidélité.

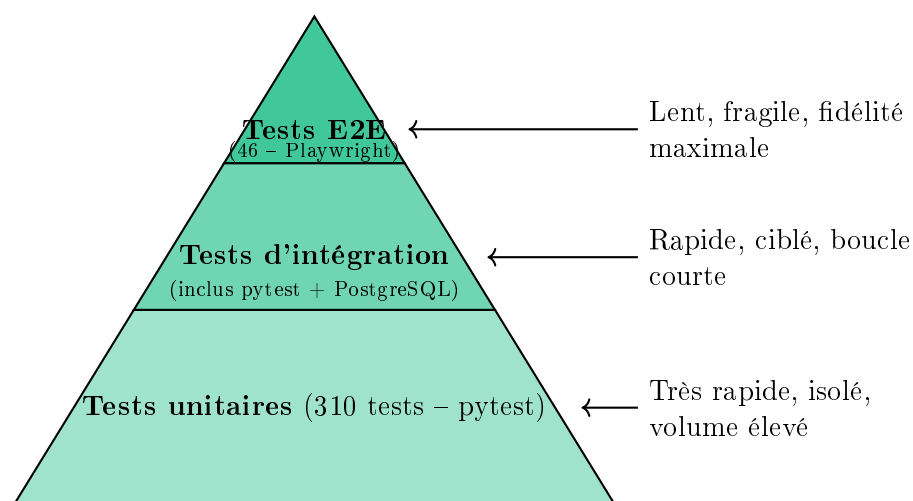


FIGURE 21 – Pyramide de tests appliquée à Vector

Le principe : plus on descend, plus les tests sont nombreux, rapides et bon marché ; plus on monte, plus ils sont fidèles au comportement réel mais coûteux. Vector applique cet arbitrage classique : 310 tests unitaires pytest exécutés en moins de trois minutes en base, complétés par une pointe de 46 tests end-to-end Playwright couvrant les sept parcours utilisateur critiques (inscription, connexion, chat, upload dataset, upload document, création corpus, isolation multi-utilisateurs).

4.2.3 Périmètre couvert

Zone fonctionnelle	Niveau de test	Couverture
Authentification et JWT	Unitaire + E2E	Complète
Gestion utilisateurs et RGD	Unitaire + E2E	Complète
Endpoints métier	Unitaire	Élevée (> 80 %)
Cascade delete	Unitaire	Complète
Streaming chat (SSE)	Unitaire (mock LLM)	Élevée
Pipeline RAG	Unitaire	Bonne
Isolation multi-utilisateurs	E2E	Complète
Interface utilisateur	E2E	Ciblée

TABLE 6 – Périmètre des tests par zone fonctionnelle

Deux zones sont volontairement peu testées automatiquement : la génération réelle par OpenAI (coût, non-déterminisme, dépendance externe fragilisante en CI, couvert par mock déterministe) et le rendu visuel exact du frontend (coût de maintenance jugé trop élevé pour le gain).

4.3 Tests unitaires backend (pytest)

4.3.1 Organisation et statistiques

Le dossier `backend/tests/` suit la structure de l'application : chaque module de l'API dispose d'un fichier de tests dédié, et les fixtures partagées sont regroupées dans un `conftest.py` accessible par tous les tests. Cette organisation facilite la navigation et isole les responsabilités : les tests décrivent leur préambule de manière déclarative, la fixture s'occupe de le construire, ce qui rend le code de test concis et lisible.

Métrique	Valeur
Nombre total de tests	310
Durée d'exécution	Moins de 3 minutes
Couverture globale du code backend	75 %
Couverture zones critiques d'authentification	100 %
Couverture des cascades de suppression	100 %
Couverture des endpoints métier	> 80 %
Tests indépendants du modèle LLM (mock)	100 %

TABLE 7 – Statistiques finales des tests unitaires backend

4.3.2 Anomalie détectée sur `coverage.py` : cas d'école méthodologique

Une anomalie particulièrement instructive a été rencontrée lors de l'audit de couverture. Les endpoints asynchrones utilisant SQLAlchemy présentaient une couverture anormalement basse (de l'ordre de 50 %), alors même que les tests exerçaient manifestement ces zones du code.

L'investigation a nécessité de comprendre le fonctionnement combiné de trois briques : SQLAlchemy 2.0 async, `coverage.py`, et le mécanisme de *greenlet* utilisé par SQLAlchemy async pour ponter le monde synchrone du driver PostgreSQL avec le monde asynchrone de l'application. Le problème s'est révélé être une limitation connue mais peu documentée : `coverage.py` ne suivait pas correctement l'exécution qui se déroulait à l'intérieur des greenlets créés par SQLAlchemy.

La résolution a consisté à ajouter une configuration ciblée dans `pyproject.toml`.

Listing 4.1 – Configuration `coverage.py` corrigeant l'anomalie

```
[tool.coverage.run]
concurrency = ["greenlet", "thread"]
```

L'effet a été immédiat : la couverture mesurée est passée de 67 % à 75 % globale, sans écrire une seule ligne de test supplémentaire. Cet épisode constitue une illustration du

principe méthodologique selon lequel il faut toujours **vérifier ses instruments avant de conclure**. Le vrai problème n'était pas l'absence de tests mais l'incapacité de l'outil de mesure à les compter correctement.

4.4 Tests end-to-end avec Playwright

4.4.1 Choix de Playwright et périmètre

Playwright a été choisi pour trois raisons principales : gestion native des attentes asynchrones (auto-wait) qui élimine une source majeure de fragilité des tests E2E, capacité à écrire les tests en TypeScript (cohérent avec le frontend Next.js), excellente ergonomie du mode debug avec inspecteur intégré. Cypress a été écarté pour ses limitations connues sur les scénarios multi-domaines et multi-onglets ; Selenium pour son API vieillissante et sa gestion peu ergonomique des timings asynchrones.

Les 46 tests couvrent sept parcours utilisateur critiques identifiés en amont (tableau 8).

Parcours	Ce qui est vérifié
Inscription utilisateur	Formulaire, validation, création en base, redirection
Connexion et déconnexion	Login, gestion du token, expiration, logout
Chat conversationnel	Envoi message, réception streamée, historique
Upload de dataset	Sélection fichier, profilage, affichage résultats
Upload de document PDF	Ingestion, découpage, indexation vectorielle
Création de corpus	Sélection documents, isolation, requêtes RAG
Isolation entre comptes	Un compte ne voit jamais les données d'un autre

TABLE 8 – Parcours utilisateur couverts par les tests end-to-end

4.4.2 Régressions détectées et mock LLM déterministe

Les tests end-to-end ont détecté au cours du stage plusieurs régressions silencieuses qui auraient probablement échappé aux tests unitaires seuls : régression sur la persistance de la sélection de thème après rechargement (inversion de deux variables lors d'un refactoring), et régression sur le format des URL de partage de conversation introduite lors du refactoring des routes Next.js pour supporter l'export statique. Ces détectations illustrent la valeur des tests E2E : ils exercent le produit dans des conditions proches de l'utilisation réelle et attrapent des régressions au niveau contractuel entre composants.

Un mock LLM déterministe a été mis en place pour découpler les tests des dépendances externes. Activable via le drapeau `USE MOCK LLM`, ce mock retourne des réponses figées, ce

qui permet aux tests end-to-end de vérifier le pipeline complet sans consommer de tokens OpenAI et sans dépendre de la disponibilité du fournisseur externe.

Bénéfice concret du mock LLM

Sans le mock, chaque exécution des 46 tests E2E aurait généré plusieurs dizaines d'appels OpenAI facturés. Sur une base d'une exécution par jour de développement pendant deux mois, cela représente une économie directe de plusieurs dizaines d'euros. Plus important : la CI aurait dépendu de la disponibilité du fournisseur externe, ce qui est incompatible avec l'objectif de reproductibilité des tests.

4.5 Intégration continue GitLab

4.5.1 Objectif et structure

Le pipeline d'intégration continue transforme les tests d'une pratique locale en une vérification systématique sur chaque *push* et *Merge Request*. Il est organisé en deux *stages* exécutés en séquence, avec parallélisation à l'intérieur de chaque stage (figure 22).

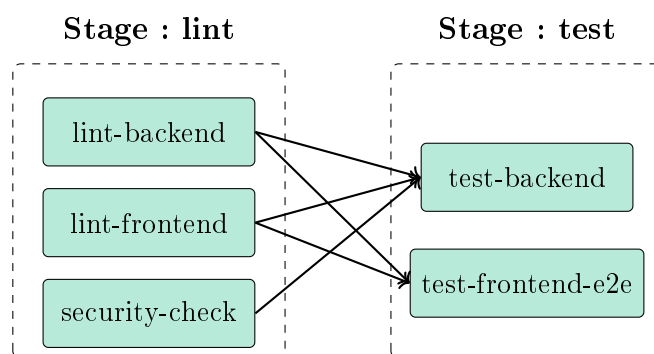


FIGURE 22 – Structure du pipeline GitLab CI

Les jobs se répartissent ainsi : **lint-backend** (Ruff sur le code Python), **lint-frontend** (ESLint TypeScript + `tsc -noEmit`), **security-check** (analyse du diff des MR à la recherche de motifs de secrets), **test-backend** (310 tests pytest dans un conteneur avec PostgreSQL + pgvector, contrôle de couverture avec seuil), **test-frontend-e2e** (46 tests Playwright en mode headless avec mock LLM).

4.5.2 Performance et incident sécurité

Le pipeline complet s'exécute en environ 7 min 39 s, grâce à la parallélisation des jobs et à l'usage de caches pour les dépendances (pip et npm).

Élément mesuré				Valeur
Durée	totale	pipeline		7 min 39 s
	complet			
Durée du stage	<code>lint</code>			~1 min 20 s
Durée du stage	<code>test</code>			~6 min 20 s
Job	le	plus	long	1 min 46 s
	(test-backend)			
Cache hit ratio	sur	les		> 90 %
	dépendances			

TABLE 9 – Métriques de performance du pipeline GitLab

Un incident significatif a marqué la mise en place de la CI : un audit rétroactif a révélé qu’une ancienne clé API OpenAI, périmée mais historiquement valide, avait été commit accidentellement dans `backend/.env.example` lors de deux commits en juin et juillet 2026. La clé était déjà épuisée en crédits au moment de l’audit, limitant le risque financier, mais une clé leakée dans Git ne peut plus jamais être considérée comme sûre.

La résolution a comporté trois actions : remplacement immédiat de la valeur incriminée par un placeholder générique ; mise en place du job `check-no-secrets` qui scanne le diff des MR et bloque le merge si un motif suspect est détecté ; décision assumée de ne **pas** réécrire l’historique Git (la clé étant épuisée, le risque était nul ; la réécriture aurait invalidé les liens vers des commits partagés). Cet incident illustre l’importance des mécanismes préventifs : un job de CI de quelques lignes suffit à rendre matériellement impossible la répétition de la même erreur.

4.6 Choix de la plateforme de déploiement

4.6.1 Contexte du pivot HuggingFace vers Render

Le choix initial s’était porté sur HuggingFace Spaces (gratuité totale historique, positionnement culturel proche de la communauté IA, support Docker). En juillet 2026, HuggingFace a modifié sa politique tarifaire [29] en rendant l’option Docker payante pour les nouveaux comptes, rendant cette solution incompatible avec la contrainte de coût nul du stage. Un pivot rapide était nécessaire.

Plateforme	Points forts	Limites identifiées
Render.com	Free tier généreux, support Docker, PostgreSQL managé, déploiement Git natif	Auto-suspension après 15 min d'inactivité
Fly.io	Excellent runtime, régions multiples	Requiert carte bancaire même en gratuit
Railway	Interface moderne	Free tier limité à 500 h/mois
Vercel	Excellent pour Next.js pur	Non adapté à un backend Python
Google Cloud Run	Scalabilité industrielle	Complexité de configuration

TABLE 10 – Comparatif des plateformes de déploiement évaluées

Render a été retenu pour la combinaison unique de gratuité complète (sans carte bancaire), support Docker de qualité industrielle, et intégration transparente avec GitLab pour l'auto-déploiement sur push. L'auto-suspension après 15 min a été jugée acceptable pour une démonstration, avec une procédure de préchauffage documentée.

4.6.2 Architecture cible du déploiement

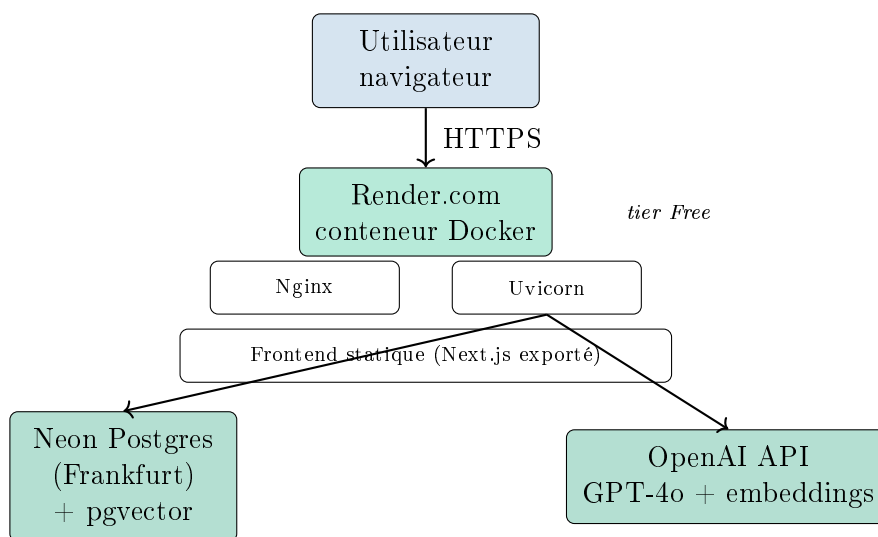


FIGURE 23 – Architecture de déploiement en production

Le conteneur Docker contient Nginx en reverse proxy interne : les requêtes vers `/api/*` sont routées vers Uvicorn (backend FastAPI), tandis que les autres requêtes servent directement le frontend Next.js pré-compilé statiquement. Cette architecture mono-conteneur présente l'avantage majeur de tenir dans les limites du tier Free Render (un seul service web autorisé).

4.7 Mise en œuvre du déploiement

4.7.1 Adaptations nécessaires et sécurisation

Le passage du développement local au déploiement mono-conteneur Docker a nécessité plusieurs adaptations : **suppression de Redis** (rate limiting via slowapi in-process, tâches planifiées via APScheduler in-process, pour tenir dans 512 MB de RAM) ; **static export Next.js** (refactoring de 4 routes dynamiques en routes basées sur paramètres de requête, car le static export ne supporte pas les routes dynamiques) ; **compte démo pré-rempli** (script de seed idempotent créant `demo@vector.ai` avec ses datasets, corpus, persona et conversations à chaque démarrage) ; **gestion du disque éphémère** (fonction de restauration des fichiers de démonstration depuis l'image Docker à chaque démarrage).

La démonstration publique est sécurisée par plusieurs mécanismes : **rate limiting slowapi** (10 chat/IP/min, 5 uploads/IP/h, 5 inscriptions/IP/jour, 20 messages/conversation/IP/h) ; **compte démo isolé** avec quotas plus permissifs mais dans les limites tier Free ; **cleanup automatique via APScheduler** (comptes invités inactifs > 48h supprimés) ; **kill switch d'urgence** (variable `DEMO_DISABLED=true` coupant en ~60s l'accès aux endpoints coûteux tout en préservant la consultation).

4.7.2 Les cinq pièges de déploiement rencontrés et résolus

Le déploiement initial n'a pas été fluide : cinq erreurs successives ont dû être diagnostiquées et corrigées. Chacune a été identifiée par la lecture attentive des logs, avec un fix ciblé et minimal, et un redéploiement pour valider.

Piège 1 : Dockerfile introuvable sur la mauvaise branche

Symptôme : `failed to read dockerfile: open Dockerfile: no such file or directory` au moment du build.

Cause : Render configuré pour déployer depuis `main`, qui n'avait pas encore reçu le Dockerfile fusionné depuis la branche de développement.

Résolution : Fusion de la MR des changements Docker vers la branche cible, puis reconfiguration de Render sur la branche `vector-agent`.

Piège 2 : Variable `DATABASE_URL` absente

Symptôme : Le conteneur démarre puis tombe sur `Errno 111 Connect call failed 127.0.0.1:5432`.

Cause : La variable `DATABASE_URL` pointant vers Neon n'avait pas été configurée dans Environment. SQLAlchemy tombait sur son fallback vers localhost.

Résolution : Ajout de la connection string Neon complète dans Environment, avec `sslmode=require`.

Piège 3 : Driver `psycopg2` incompatible avec l'`asyncio`

Symptôme : `sqlalchemy.exc.InvalidRequestError: The asyncio extension requires an async driver. The loaded 'psycopg2' is not async.`

Cause : L'URL `postgresql://...` de Neon interprétée par défaut comme `postgresql+psycopg2://` (driver synchrone), incompatible avec

`async_engine_from_config` utilisé par Alembic.

Résolution : Normalisation d'URL dans `migrations/env.py` forçant `postgresql+asyncpg` juste avant la création du moteur.

Piège 4 : Paramètres SSL incompatibles avec `asyncpg`

Symptôme : `TypeError: connect() got an unexpected keyword argument 'channel_binding'` puis `'sslmode'`.

Cause : Paramètres SSL modernes ajoutés par Neon (`channel_binding=require`, `sslmode=require`) sont des conventions `libpq/psycopg2` non reconnues par `asyncpg` qui utilise une API SSL native distincte.

Résolution : Extension du `fix` précédent pour retirer les paramètres SSL `libpq` et passer `connect_args={"ssl": "require"}` via l'API native `asyncpg`, conditionnellement selon l'hôte (préserver le fonctionnement local).

Piège 5 : UUID hardcodée dans fixture obsolète

Symptôme : Après un déploiement réussi, connexion `demo@vector.ai` refusée avec « Email ou mot de passe incorrect ».

Cause : Le seed du compte démo échouait sur `ForeignKeyViolationError` car un `agent_id` figé dans la fixture ne correspondait pas à l'agent Vector réellement en base (UUID régénéré à chaque premier boot). L'entrypoint continuait silencieusement.

Résolution : Remplacement de la lecture de l'`agent_id` depuis la fixture par une lecture dynamique via `SELECT`. Une donnée absente honnête vaut mieux qu'une donnée présente périmée.

Ces cinq pièges ont été résolus en cinq itérations. La discipline méthodologique clef : **ne jamais deviner, toujours lire les logs**. Les logs disent la vérité ; les hypothèses formulées sans lecture des logs se révèlent presque toujours partiellement fausses.

4.8 Mesures de performance en production

4.8.1 Méthodologie de mesure

Une fois le déploiement stable, une phase de mesure de performance a été menée pour caractériser objectivement le comportement en production. Un script Python autonome a été développé, s'appuyant sur `httpx` pour les requêtes réalistes et sur le module `statistics` pour les percentiles.

Cinq métriques ont été mesurées : **cold start** (temps de réponse après inactivité prolongée), **temps de réponse warm** (latence en régime nominal), **latence de login** (authentification bout-en-bout), **livraison statique** (frontend via Nginx), et **disponibilité baseline** (pourcentage de requêtes réussies sur fenêtre courte).

4.8.2 Résultats obtenus

Métrique	Min	Médiane	p95	p99	Max
Cold start	–	–	–	–	82.4 s
Warm /health (30x)	62.5 ms	74.8 ms	320.0 ms	426.4 ms	428.6 ms
Login (10x)	1594.5 ms	1681.3 ms	2341.7 ms	–	2643.5 ms
Static / (10x)	61.1 ms	68.3 ms	116.3 ms	–	130.2 ms

TABLE 11 – Mesures de performance en production

La disponibilité mesurée sur une fenêtre courte de 60 secondes atteint 100 % (30/30).

4.8.3 Analyse et interprétation

Le **cold start de 82 s** traduit la contrainte du tier Free Render : la reprise nécessite de redémarrer le conteneur, ré-établir les connexions à Neon, et attendre que le premier health check passe. Ce coût est acceptable en démonstration avec un préchauffage préalable.

Le **login à 1.68 s** peut sembler lent à première vue, mais c'est en réalité un signal positif de sécurité : il indique un hashage bcrypt avec un facteur de coût suffisant pour rendre les attaques par force brute coûteuses. Un login instantané serait au contraire un signal négatif, révélateur d'un hachage trop faible. La latence inclut l'aller-retour TLS avec Neon (Frankfurt), la vérification du mot de passe, la génération et signature du JWT.

Les **chiffres warm** (75 ms médiane health, 68 ms static, p95 raisonnables) sont cohérents avec un service correctement déployé sur un tier Free avec 0.1 vCPU. Les valeurs plus élevées sur les percentiles hauts sont attribuables au bruit réseau et à la contention CPU occasionnelle du tier partagé.

4.9 Démonstration publique livrée

4.9.1 URL et compte pré-rempli

La démonstration est accessible publiquement à l'adresse suivante :

<https://ai-commandos-multi-agent.onrender.com>

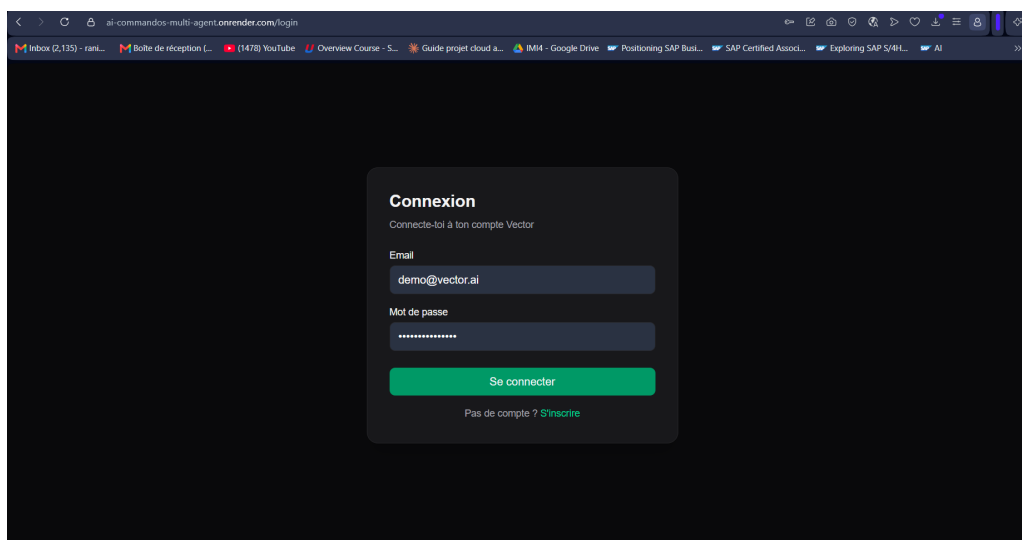


FIGURE 24 – Page d'accueil de la démonstration publique de Vector

Un compte de démonstration pré-rempli permet à tout visiteur d'expérimenter immédiatement toutes les fonctionnalités : email `demo@vector.ai`, mot de passe `VectorDemo2026!`. Le compte contient un dataset (CSV pipeline commercial IBM fictif), un corpus (rapport stratégique PDF et livre blanc DOCX fictifs mais réalistes), un persona spécialisé (« Vector Analyste Finance IBM »), et trois conversations d'exemple.

4.9.2 Enseignements sur le déploiement

Le processus de déploiement a laissé plusieurs enseignements structurants. **Le déploiement est toujours plus long que prévu** : chaque piège individuel semblait mineur, mais leur accumulation a représenté une part significative du temps total. **La lecture attentive des logs est la compétence la plus rentable en déploiement** : les hypothèses formulées sans lecture des logs se sont presque toutes révélées partiellement fausses. **La valeur des mécanismes préventifs** (kill switch, rate limiting, cleanup) devient tangible dès qu'on expose un service publiquement. **L'existence d'un compte de démonstration pré-rempli change complètement l'expérience du visiteur** : sans lui, un jury doit s'inscrire, uploader, attendre le profilage ; avec lui, la valeur ajoutée s'apprécie immédiatement.

4.10 Conclusion

Ce chapitre a présenté l'ensemble des efforts qualité et de déploiement qui ont accompagné et complété le développement fonctionnel de Vector. Sur le plan de la qualité, la pyramide de tests bien dimensionnée (310 unitaires, 46 E2E, 75 % de couverture) et le pipeline CI rapide (7 min 39 s) fournissent un socle industriel de confiance. Sur le plan du déploiement, le pivot HuggingFace vers Render, l'architecture mono-conteneur adaptée au tier Free, la résolution méthodique des cinq pièges rencontrés, et la mise en œuvre des protections de la démonstration publique ont abouti à un service en ligne fonctionnel et sécurisé, mesuré objectivement à travers cinq catégories de métriques.

La démonstration publique livrée à l'adresse <https://ai-commandos-multi-agent.onrender.com> est le résultat concret et vérifiable de l'ensemble du travail présenté dans

ce rapport. Elle constitue à la fois le livrable central du stage et la preuve tangible que Vector n'est pas resté à l'état de prototype local mais a bien franchi le cap de la mise en production.

Conclusion générale

Le stage présenté dans ce rapport avait pour ambition de concevoir, développer et déployer, de manière autonome et complète, un assistant d'intelligence artificielle d'entreprise nommé Vector. À l'issue de cette démarche, il est possible de dresser un bilan structuré sur trois plans complémentaires : les livrables techniques réalisés, les compétences acquises, et les enseignements personnels tirés de cette expérience.

Sur le plan des livrables techniques, le projet a atteint l'ensemble des objectifs initialement fixés. Vector est aujourd'hui un produit complet, combinant une architecture backend en *FastAPI* et frontend en *Next.js*, une base de données *PostgreSQL* enrichie de l'extension *pgvector* pour la recherche sémantique, et une intégration native avec les modèles d'*OpenAI*. Sept modules fonctionnels majeurs ont été développés : le chat conversationnel multimodal, l'ingestion et l'analyse de données tabulaires, l'interrogation documentaire par RAG, la constitution de corpus thématiques, les assistants personnalisés (personas), la génération automatique de tableaux de bord, et un ensemble d'éléments d'expérience utilisateur transverses (thème, paramètres, RGPD). Cet ensemble est validé par plus de 310 tests unitaires backend et 46 tests end-to-end *Playwright*, avec une couverture de 75% mesurée par un audit rigoureux. L'intégration continue *GitLab CI/CD* exécute automatiquement l'ensemble de ces vérifications à chaque modification en moins de 8 minutes. Enfin, le déploiement final sur la plateforme *Render.com*, avec une base de données externalisée sur *Neon*, rend Vector accessible publiquement via une URL stable.

Sur le plan des compétences acquises, ce stage a permis de consolider et d'étendre significativement un ensemble de compétences techniques et méthodologiques. Sur le versant technique, la maîtrise complète d'une stack moderne (Python asynchrone, TypeScript typé, PostgreSQL avec extensions, Docker, CI/CD) constitue un socle transférable à de nombreux contextes professionnels. La compréhension approfondie des mécanismes de l'intelligence artificielle générative — notamment le pipeline RAG, la génération d'embeddings, le streaming de réponses, la gestion des prompts système — ouvre des perspectives dans un domaine en forte croissance. Sur le versant méthodologique, la conduite d'un projet en autonomie sur plusieurs mois, avec la discipline associée (tests, documentation, versioning, itérations courtes), l'usage raisonné d'outils d'assistance IA au développement, et la capacité à s'adapter aux imprévus techniques constituent des compétences dont l'importance dépasse largement le cadre de ce stage spécifique.

Sur le plan des difficultés rencontrées et de leur traitement, plusieurs épisodes ont ponctué le stage. Le pivot de *HuggingFace Spaces* vers *Render.com*, suite au changement de politique tarifaire de HuggingFace en juillet 2026, a démontré l'importance de la capacité d'adaptation face à un environnement technologique évolutif. Les quatre régressions silencieuses détectées lors de la première exécution complète des tests end-to-end ont illustré la valeur de l'automatisation dans un projet mature. L'anomalie identifiée sur l'outil *coverage.py* dans le contexte *SQLAlchemy* asynchrone a

rappelé l'importance de vérifier ses instruments de mesure avant de tirer des conclusions. Enfin, l'audit sécurité proactif qui a révélé une dette de sécurité héritée du démarrage du projet a permis de mettre en place des garde-fous techniques (job CI de vérification des secrets) qui bénéficieront au projet à long terme.

Sur le plan des perspectives d'évolution, plusieurs axes prometteurs se dégagent naturellement des travaux réalisés :

- **Support multi-tenants** : évolution du modèle de données pour supporter des organisations avec plusieurs utilisateurs partageant des ressources communes, tout en préservant les cloisonnements de sécurité.
- **Diversification des modèles LLM** : intégration d'un choix de modèles (Claude, Gemini, modèles open source) permettant à l'utilisateur ou à l'administrateur de basculer selon les cas d'usage.
- **Amélioration des capacités RAG** : mise en place de techniques avancées comme le *hybrid search* (BM25 combiné aux embeddings), le *reranking* par cross-encoder, ou la *query expansion* pour améliorer la pertinence des retours.
- **Persistance des fichiers en production** : externalisation vers un stockage objet (S3-compatible, R2, B2) pour préserver les documents uploadés au-delà du cycle de vie du conteneur.
- **Intégration MCP (Model Context Protocol)** : exposition des ressources Vector via le protocole MCP émergent, permettant à d'autres assistants IA de consommer les données Vector en toute sécurité.
- **Analytics d'usage** : mise en place d'un tableau de bord d'administration présentant les statistiques d'utilisation, la consommation de tokens, et les erreurs pour piloter finement l'exploitation.

Sur le plan des enseignements personnels, ce stage restera une expérience formatrice à plusieurs titres. Il a permis de vivre concrètement la réalité d'un projet mené de bout en bout, avec l'ensemble de ses phases : conception initiale, développement itératif, adaptations aux imprévus, phase de consolidation qualité, préparation au déploiement, et livraison publique. Il a également confirmé l'intérêt d'une approche méthodique et documentée, où chaque décision est tracée et argumentée, ce qui facilite grandement les phases ultérieures de reprise, de communication et de valorisation. Enfin, il a mis en évidence la valeur d'un usage réfléchi des outils d'assistance IA au développement : ces outils ne remplacent pas la conception et le jugement humains, mais permettent d'accélérer significativement la production de code et d'explorer plus systématiquement les alternatives techniques.

En définitive, ce projet démontre qu'il est aujourd'hui possible, avec des ressources maîtrisées et une méthodologie rigoureuse, de concevoir et de déployer un assistant IA d'entreprise complet, techniquement solide, et prêt à être présenté à un public exigeant. Vector constitue à la fois un livrable technique tangible et une preuve de concept d'une démarche de développement industriel accessible aux étudiants ingénieurs bien formés. Les perspectives d'évolution identifiées ouvrent la voie à des développements futurs qui pourront prolonger et amplifier la valeur produite pendant ce stage.

Bibliographie

- [1] McKinsey & Company. (2024) The State of AI in 2024. En ligne : <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai>.
- [2] Gartner. (2024) Emerging Tech : Generative AI — The Business Impact Curve. En ligne : <https://www.gartner.com/en/topics/generative-ai>.
- [3] Sebastian Ramírez. (2026) FastAPI — Modern, fast (high-performance), web framework for building APIs with Python. En ligne : <https://fastapi.tiangolo.com/>.
- [4] SQLAlchemy Development Team. (2026) SQLAlchemy 2.0 Documentation. En ligne : <https://docs.sqlalchemy.org/en/20/>.
- [5] Pydantic Team. (2026) Pydantic — Data Validation using Python Type Hints. En ligne : <https://docs.pydantic.dev/>.
- [6] Vercel Inc. (2026) Next.js Documentation. En ligne : <https://nextjs.org/docs>.
- [7] Tailwind Labs Inc. (2026) Tailwind CSS Documentation. En ligne : <https://tailwindcss.com/docs>.
- [8] OpenAI. (2026) OpenAI API Reference. En ligne : <https://platform.openai.com/docs/api-reference>.
- [9] ——. (2024) Hello GPT-4o. En ligne : <https://openai.com/index/hello-gpt-4o/>.
- [10] ——. (2026) OpenAI Embeddings Guide — text-embedding-3-small. En ligne : <https://platform.openai.com/docs/guides/embeddings>.
- [11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, en ligne : <https://arxiv.org/abs/2005.11401>.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, en ligne : <https://arxiv.org/abs/1706.03762>.
- [13] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, en ligne : <https://arxiv.org/abs/2005.14165>.
- [14] The PostgreSQL Global Development Group. (2026) PostgreSQL 16 Documentation. En ligne : <https://www.postgresql.org/docs/16/>.
- [15] Andrew Kane. (2026) pgvector — Open-source vector similarity search for Postgres. En ligne : <https://github.com/pgvector/pgvector>.
- [16] Docker Inc. (2026) Docker Documentation. En ligne : <https://docs.docker.com/>.
- [17] Nginx Inc. (2026) Nginx Documentation. En ligne : <https://nginx.org/en/docs/>.

- [18] GitLab Inc. (2026) GitLab CI/CD Documentation. En ligne : <https://docs.gitlab.com/ee/ci/>.
- [19] pytest development team. (2026) pytest — helps you write better programs. En ligne : <https://docs.pytest.org/>.
- [20] Microsoft Corporation. (2026) Playwright — Fast and reliable end-to-end testing for modern web apps. En ligne : <https://playwright.dev/>.
- [21] R. C. Martin, *Clean Code : A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [22] Render Services Inc. (2026) Render Documentation. En ligne : <https://render.com/docs>.
- [23] Neon Inc. (2026) Neon Documentation — Serverless Postgres. En ligne : <https://neon.tech/docs>.
- [24] Internet Engineering Task Force (IETF). (2015) RFC 7519 — JSON Web Token (JWT). En ligne : <https://datatracker.ietf.org/doc/html/rfc7519>.
- [25] OWASP Foundation. (2024) OWASP Top 10 — Web Application Security Risks. En ligne : <https://owasp.org/www-project-top-ten/>.
- [26] pgvector Project. (2026) IVFFlat indexing in pgvector. En ligne : <https://github.com/pgvector/pgvector#ivfflat>.
- [27] Commission Nationale de l'Informatique et des Libertés (CNIL). (2024) Comprendre le RGPD. En ligne : <https://www.cnil.fr/fr/comprendre-le-rgpd>.
- [28] M. Fowler, *Refactoring : Improving the Design of Existing Code*, 2nd ed. Addison-Wesley Professional, 2018.
- [29] Hugging Face. (2026) Hugging Face Spaces Pricing Update, July 2026. En ligne : <https://huggingface.co/pricing>.